2026 年 NUDT-CSU 数学建模联赛

承　诺　书

我们完全明白，在竞赛开始后参赛队员不能以任何方式（包括电话、电子邮件、网上咨询等）与本队以外的任何人（包括指导教师）研究、讨论与赛题有关的问题。

我们知道，抄袭别人的成果和买卖论文、代码都是违反竞赛规则的，如果引用别人的成果或其它公开的资料（包括网上查到的资料），必须按照规定的参考文献的表述方式在正文引用处和参考文献中明确列出；按要求使用AI开展竞赛。

我们郑重承诺，严格遵守竞赛规则，以保证竞赛的公正、公平性。如有违反竞赛规则的行为，我们愿意承担由此引起的一切后果。

所属学校（学校全称）：中南大学

参赛队员：

队员 1 姓名：殷岳　　学号：

队员 2 姓名：赵可欣　　学号：

队员 3 姓名：骆笑　　学号：

日期：2026 年 6 月 23 日

附件 2：

2026 年 NUDT-CSU 数学建模联赛

题　目：　　基于多尺度谱估计与鲁棒传感器融合的机械设备故障检测研究

摘　要：

机械设备早期微弱故障信号远弱于运行噪声，多源频率叠加与传感器受限增大识别难度。本文基于 100Hz 采样、400s 振动时序数据，建立微弱故障检测、波形恢复、多源信号分离与传感器鲁棒布局递进模型。

针对问题一：单源弱故障检测需兼顾频谱分辨能力与估计稳定性。全段 FFT 频率分辨率更高，但单次周期图波动幅度较大；Welch 分段平均谱能够降低噪声方差，却会损失部分分辨率。本文**结合两种谱估计方式构建联合检测模型，将单一谱分析改为双向互补校验，引入最大谱线检验控制全频段虚警概率，确定故障特征频率为 2.0000Hz**。

针对问题二：已知故障频率后重构波形，直接对正弦信号开展非线性搜索易受初值、相位周期特性影响，极易出现局部收敛问题。本文**利用三角恒等变换将求解过程转化为线性最小二乘参数估计，通过正交基线性回归完成解析计算，得到故障振幅 0.0352、初相 1.574rad，残差均方根 0.1000**；依托**残差重采样仿真**开展重复性测试，**波形归一化误差中位数为 4.47%**，验证波形恢复效果。

针对问题三：多故障源信号分离需要自动判断源数量，同时削弱不同分量之间的频谱干扰。阈值设置过严会遗漏幅值较弱的故障分量，逐一分量拟合则容易引发频谱泄漏。本文**依次完成候选频率筛选、借助 BIC 准则自动确定模型阶数、采用多频联合最小二乘实现信号拆分**，同步解决源数未知与参数耦合难题，**最终确定最优故障源数量为 4，分离得到 4.0000Hz、7.9999Hz、13.0001Hz、14.0000Hz 四组故障分量**；通过**相邻双频分辨数值实验**证明，**该模型能够稳定区分间距 0.005Hz 及以上的邻近故障频率**。

针对问题四：传感器布设优化核心是在测点数量有限的前提下，提升所有故障源中最难检测分量的识别效果。综合振动传播衰减规律、测点空间噪声差异

以及系统虚警率约束条件，**先计算单测点信噪比，再通过非相干能量融合整合多测点数据，搭建以 max-min 最差源检测概率为优化目标的鲁棒布局模型**。在典型算例中采用三点分散布设方案，**系统达到 0.9 检测概率的所需时长由单传感器 220.7s 缩短至 64.4s**；利用**蒙特卡洛仿真校验，系统实际虚警率和预设 0.01 标准保持一致**。

最后，本文对整套模型进行综合评估：模型贴合工业设备监测实际工况，可有效解决微弱故障识别、多分量信号分离以及传感器优化布设相关问题，具备较强**实用性与计算效率**，同样能够应用于旋转机械故障诊断、结构健康监测、工业设备状态监测等相关领域。

关键词：微弱故障检测；Welch 谱估计；BIC 模型选择；联合最小二乘；传感器鲁棒布局。

目录

一、 问题重述 1

1.1 问题背景 1

1.2 问题提出 1

1.3 资料条件与本文工作 1

二、 问题分析 2

2.1 问题一的分析 2

2.2 问题二的分析 2

2.3 问题三的分析 3

2.4 问题四的分析 3

三、 模型假设 3

四、 符号说明 4

五、 数据预处理 5

5.1 时域信号核验 5

5.2 频域数据规约 6

5.3 空间几何数据构造 6

六、 模型建立 6

6.1 问题一：单源故障频率检测模型 6

6.2 问题二：单源故障波形恢复模型 8

6.3 问题三：多故障源自动分离模型 8

6.4 问题四：传感器融合检测与布局优化模型 9

七、 模型结果与分析 10

7.1 问题一结果：单源频率检测 10

7.2 问题二结果：波形恢复与误差评价 12

7.3 问题三结果：多故障源分离 14

7.4 问题四结果：传感器布局优化 16

八、 模型检验 18

8.1 衰减系数灵敏度 18

8.2 理论分布与蒙特卡洛核验 19

8.3 积累时长影响 20
8.4 虚警率约束灵敏度 20
8.5 消融实验与可复现性检查 21
九、 模型评价与改进 22
十、 结论 22
参考文献 22
附录：支撑材料与完整程序 24

插图清单

图 1　总体技术路线 2
图 2　单源故障信号时频分析 11
图 3　频谱分析方法性能对比 11
图 4　单源故障信号恢复与残差 12
图 5　信号恢复结果的频域对比 13
图 6　多源频谱寻峰与 BIC 选阶 14
图 7　多故障源时域波形分离 15
图 8　相邻双频分辨实验 15
图 9　问题四传感器布局优化流程图 16
图 10　代表性算例的鲁棒布局 16
图 11　系统 ROC 与故障功率敏感性 17
图 12　理论值与蒙特卡洛核验 19
图 13　积累时长对检测概率的影响 20
图 14　虚警率约束下的检测性能 21

附表清单

表 1　主要符号说明 5
表 2　时域数据预处理流程 5
表 3　三种频谱分析方法定量对比 11
表 4　单源故障波形恢复结果 13
表 5　单源波形有真值仿真误差（500 次） 13
表 6　多故障源联合最小二乘估计结果 14
表 7　代表性算例的鲁棒三传感器布局 17
表 8　传感器数量对检测性能的影响 17
表 9　衰减系数灵敏度 19
表 10　不同虚警率约束下最差故障源检测概率 20
表 11　模型关键模块消融分析 21
表 12　支撑材料文件清单 24

一、 问题重述

1.1 问题背景

旋转机械设备在长期运行中会因齿轮磨损、轴承裂纹、啮合冲击等原因产生周期性微弱振动，并已成为故障诊断研究的重要对象[1]。此类信号的能量通常远低于电机噪声、结构振动和测量噪声形成的背景基底，若仅在时域观察，早期故障特征会被完全淹没，导致传统阈值报警容易出现漏检或误报。

从信号处理角度看，故障分量虽弱，但是它的能量具有高度集中的频率结构，而噪声近似宽带分布。近年来，多变量信号处理也被用于复杂工况下的旋转机械故障诊断[1]。因而，若能在频域中提高谱估计分辨率并降低估计方差，就可以把“看不见”的时域周期在功率谱上显影。本文据此将故障检测建模为统计检测与参数估计问题，并进一步延伸到多源分离和传感器空间布局。

1.2 问题提出

题目给定的核心观测关系为：传感器采集信号 $x(t)$ 由故障周期信号 $s(t)$ 与背景噪声 $n(t)$ 线性叠加而成。本文需要在单源和多源两类场景下完成频率检测、波形恢复、源数判定与传感器优化布局。具体任务可归纳为以下四个层次：

问题 1： 在单源弱故障条件下，从强噪声时域序列中检测故障特征频率 f_0，并说明所用方法相对于传统傅里叶分析的优势。

问题 2： 在 f_0 已知基础上估计故障信号振幅 A 与初相 φ_0，恢复完整波形，并从时域和频域量化误差。

问题 3： 在故障源数量未知时，自动识别多个故障频率并估计各分量参数，同时讨论频率分辨极限。

问题 4： 基于设备几何结构和故障源位置，设计最多 3 个传感器的布局，使检测概率尽可能高、误报率尽可能低。

1.3 资料条件与本文工作

附件提供单源与多源两组单通道时间序列，采样率 $f_s = 100\,\mathrm{Hz}$，均含 $N = 40001$ 个采样点，对应观测时长 $T = 400\,\mathrm{s}$。题目未给设备几何尺寸、故障源坐标、传播衰减系数及测点噪声场，因此问题四先建立不依赖具体几何的通用模型，再用明确标注的代表性场景说明求解流程。围绕这些数据，本文工作路线如图 1 所示。

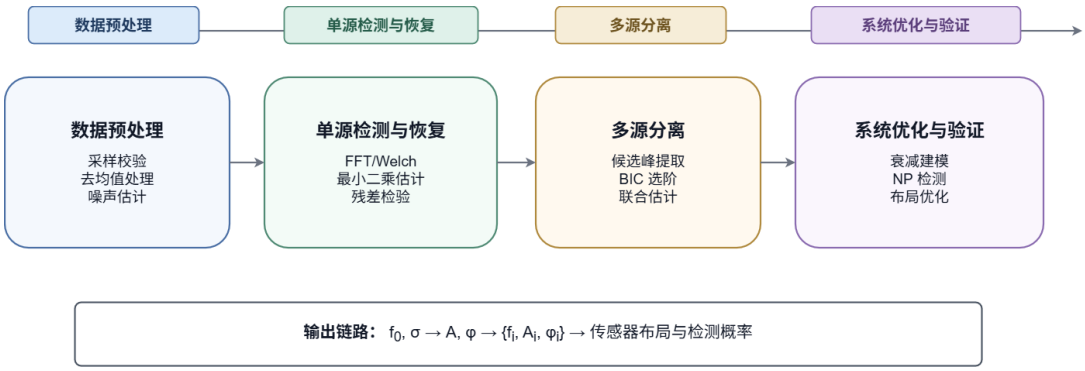


图 **1**　总体技术路线

本文把四问组织成“前问输出作为后问输入”的闭环：问题一给出 f_0 与噪声基底，问题二给出单源振幅与相位，问题三给出多源数量、频率与振幅，问题四再将这些结果转化为传感器布局模型的输入。

二、 问题分析

2.1 问题一的分析

单源检测的困难并不在于是否存在傅里叶峰值，而在于弱峰能否与随机噪声起伏可靠区分。FFT 可视为把时域序列投影到一组不同频率的复指数基上：周期故障分量在对应频率处相干叠加，宽带随机噪声则分散于各谱线。因此，全长序列可凭借较小的频率间隔 $\Delta f = f_s/N$ 精确定位谱峰，但单次周期图的方差较大，噪声也可能形成偶然尖峰。Welch 方法将数据分成相互重叠的短段，对各段加 Hann 窗后分别计算周期图并取平均；加窗用于抑制有限截断引起的频谱泄漏，分段平均则抵消不同数据段中的随机起伏，以一定的频率分辨率损失换取更稳定的谱估计。

基于上述互补关系，本文采用“FFT 定位、Welch 复核、统计阈值判决”的联合流程：先利用全长 FFT 形成候选频率，再检查该峰是否在 Welch 平均谱中仍稳定存在，最后用 Bonferroni 校正控制遍历全频段带来的总体虚警率。这样既保留长序列的精细定位能力，又避免仅凭单次 FFT 最大峰作出判断；其改进并非提高 FFT 的理论分辨率，而是同时增强弱峰识别的稳定性并给出可量化的全频段误报约束。

2.2 问题二的分析

已知故障频率后，待求量仅为振幅与初相。若直接对正弦形式进行非线性搜索，结果会受到初值和相位周期性的影响。利用正弦、余弦两组正交基可把该问题改写成线性回归，再由两个回归系数还原振幅和相位。模型是否有效不能只看

拟合曲线，还需比较残差均方根、频域峰值和参数区间，因此本文从时域与频域两方面进行误差检验。

2.3 问题三的分析

多源情形同时包含“源数未知”和“参数耦合”两类困难。阈值过严会遗漏弱分量，阈值过松又会引入噪声峰；逐个拟合还可能使相邻频率相互泄漏。本文先适度放宽寻峰条件形成候选集，再以信息准则在拟合误差与模型复杂度之间折中，确定源数后统一构建设计矩阵，使所有频率分量在同一次最小二乘计算中完成校正。本问所称“定位”是故障特征在频率轴上的定位；附件仅含单通道数据，缺少多测点传播信息，因而不能据此唯一确定故障源的物理空间位置。

2.4 问题四的分析

传感器布局既要靠近故障源以获得较强信号，也要避免多个测点集中在局部区域而造成覆盖失衡；相关研究表明，安装不确定性、测点聚集和优化目标选择都会影响布点质量[2, 3]。因为虚警率由报警制度预先约束，布局优劣可转化为固定系统总虚警率下检测概率的比较。本文用距离衰减描述传播损失，以噪声归一化的非相干能量融合汇总独立测点信息[4]，并采用 max-min 准则优先改善最难检测的故障源，最终在可安装网格上搜索可执行方案[5]。

三、 模型假设

为简化问题并保证模型具有明确的统计意义，本文作出如下假设：

假设 1： **噪声局部平稳且条件高斯：**预处理后，背景噪声在分析窗内近似为零均值高斯弱平稳过程，目标频带内无无法辨别的强同频干扰。该条件为问题一的阈值、问题二和三的参数估计及问题四的卡方分布提供依据，并由残差正态性和自相关核验；若失配，则进行协方差预白化，并以稳健谱或残差 Bootstrap 标定阈值。

假设 2： **故障响应线性叠加：**在传感器线性工作区间内，观测量等于各故障周期分量与噪声之和。该条件支撑问题一至三的正交分解和最小二乘估计，也使问题四可分源计算信噪比；若残差存在显著谐波、调制边带或幅值相关结构，应增加非线性项或改用非线性系统辨识。

假设 3： **工况与参数窗内稳定：**设备在观测窗内近似恒速，各分量的频率、振幅和相位不发生突变，因而四问可使用定频正弦模型并传递参数；若转速或幅值明显漂移，应缩短分析窗，采用同步重采样、阶次谱或时变参数模型。

假设 4： **采样充分且时间基准准确**：采样间隔稳定，采样率满足奈奎斯特条件，$0 \sim 20\,\mathrm{Hz}$ 频段不存在显著混叠。该条件保证问题一至三的频率轴、相位和分辨率有效；若欠采样或时钟失准，应先进行抗混叠采集、时间校准或重采样。

假设 5： **多测点噪声条件独立**：给定故障响应后，不同传感器噪声近似相互独立，使问题四可采用非相干能量融合并累加信噪比；若健康数据表明测点间显著相关，则估计完整噪声协方差矩阵，经空间预白化后重算联合统计量与阈值。

假设 6： **空间传播参数在检测期内稳定**：问题四中，各测点的传播增益和噪声尺度近似不变，距离衰减可作为缺少标定数据时的代理模型，使布局能统一比较；若存在反射、多路径或局部共振，应以试验频率响应 $H_{ij}(f_j)$ 替代指数衰减关系。

算例条件说明：问题四另取 $4\,\mathrm{m} \times 2\,\mathrm{m}$ 的代表性区域并按 $0.25\,\mathrm{m}$ 离散。几何范围、网格、源位置和传播参数仅为可复现算例设置，并非题目事实或普遍假设；工程应用时须以真实图纸、可安装区域和现场标定结果替换后重新寻优。

四、 符号说明

全文主要符号及单位见表 1，其余局部变量在首次出现时说明。

表 **1**　主要符号说明

符号	含义	单位/说明
$x(t), x[k]$	传感器采集的连续/离散振动加速度信号	a.u.
$s(t)$	故障齿轮产生的弱周期信号	—
$n(t)$	背景噪声	—
A, A_i	单源/第 i 个故障分量振幅	a.u.
f_0, f_i	单源/第 i 个故障特征频率	Hz
φ_0, φ_i	单源/第 i 个故障分量初相	rad
f_s, N, T	采样率、采样点数、观测时长	Hz、点、s
Δf	频率分辨率，$\Delta f = f_s/N$	Hz
P_{FFT}, P_W	FFT 功率谱与 Welch 功率谱估计	—
L, K, B	分段长度、分段数、独立搜索频点数	点、段、个
M	故障源数量	个
BIC	贝叶斯信息准则，用于模型阶数选择	—
d_{ij}	第 i 个传感器到第 j 个故障源的距离	m
α	信号传播的指数衰减系数	m^{-1}
$\rho_{ij}, \rho_{\mathrm{eq}}^{(j)}$	单传感器/融合等效信噪比	—
$P_{FA}^{\mathrm{sys}}, P_{FA}^{(1)}, P_D$	系统总虚警率、单频虚警率、检测概率	—
g_{ij}, σ_i	测点灵敏度系数、位置相关噪声标准差	—
N_o, q	检测窗样本数、选用传感器数	点、个
λ_j, γ_q	非中心参数、检测阈值	—
$\Omega, \mathcal{S}$	候选安装点集合、所选传感器集合	—
RSS_M, k_M	M 阶模型残差平方和、参数个数	—

五、 数据预处理

5.1 时域信号核验

首先读取附件中的时间序列，按表 2 核验采样间隔并去均值。单源数据均值为 4.467×10^{-5}，标准差为 0.103063；多源数据标准差为 0.106421。两表均有 40001 个样本，时间从 0 到 400 s，不存在把“样本点数”误写成“采样区间数”的问题。

表 **2**　时域数据预处理流程

项目	处理方式	目的
采样率核验	由相邻时间戳差分估计 $f_s = 100\,\mathrm{Hz}$	确认频率轴与奈奎斯特条件
去均值	$x[k] \leftarrow x[k] - \mathrm{mean}(x)$	削弱直流峰对谱估计的影响
异常检查	检查缺失、重复与极端跳点	保证 FFT/Welch 不受非物理点污染
短时可视化	绘制前 5 s 时域波形	说明时域弱故障不可直接辨识

5.2 频域数据规约

频域处理的关键是根据问题目标选择合适的谱估计尺度[6, 7]。单段全信号 FFT 保留最高频率分辨率，适合精确定位单频谱峰；Welch 方法通过分段加窗平均降低谱估计方差，适合验证峰值稳健性。本文将两者配合使用，避免“单次 FFT 偶然峰值”与“短窗频率分辨率不足”两个常见问题。

5.3 空间几何数据构造

题目没有给出可直接计算坐标的设备图纸。为展示通用模型的数值求解，本文另设代表性矩形区域 $[0,4]\times[0,2]$，将四个源放在 $(0.5,1.0)$、$(1.5,0.3)$、$(2.5,1.7)$、$(3.5,1.0)$。仅取边界内侧的 $0.25\,\mathrm{m}$ 网格，得到 105 个候选点；三传感器组合数为

$$C(105,3)=187460,$$

规模适中，可直接穷举获得该代表性场景下的离散全局最优解。源坐标、区域尺寸、衰减系数和噪声场均属于算例设定，不当作题设条件。

六、 模型建立

6.1 问题一：单源故障频率检测模型

问题一是在强噪声下进行单频率统计检测。离散观测模型为：

$$x[k]=A\sin(2\pi f_0 k\Delta t+\varphi_0)+n[k],\quad k=0,1,\ldots,N-1. \tag{1}$$

对 $x[k]$ 作离散傅里叶变换，得到频域表示：

$$X[m]=\sum_{k=0}^{N-1}x[k]e^{-j2\pi mk/N},\quad f_m=m\frac{f_s}{N}. \tag{2}$$

定义归一化单边功率谱：

$$P_{\mathrm{FFT}}[m]=\left|\frac{X[m]}{N}\right|^2,\quad 0\le m\le\left\lfloor\frac{N}{2}\right\rfloor. \tag{3}$$

当信号中存在振幅为 A 的纯正弦分量时，其能量集中在 f_0 附近，而噪声能量分散在频率轴上，因此谱峰与噪声基底的理论比值近似为：

$$\mathrm{SNR}_{\mathrm{spec}}\approx\frac{(A/2)^2}{\sigma_n^2/N}=\frac{NA^2}{4\sigma_n^2}. \tag{4}$$

式 (4) 直接解释了长时间采样对弱信号检测的优势：N 越大，单频信号在频域累积得越突出。为降低单段 FFT 的随机波动，本文进一步采用 Welch 分段平均谱[8]：

$$P_W(f) = \frac{1}{K}\sum_{i=1}^{K}\frac{1}{f_s W_{ss}}\left|\sum_{l=0}^{L-1} x_i[l]w[l]e^{-j2\pi fl/f_s}\right|^2. \tag{5}$$

其中，L 为每段样本数，K 为参与平均的分段数，$w[l]$ 为 Hann 窗，$W_{ss} = \sum_{l=0}^{L-1} w^2[l]$ 用于校正窗函数造成的能量尺度变化。若各段周期图近似独立，同一频点的平均可显著减小估计方差；采用 50% 重叠则能在加窗后提高数据利用率。由于分段使频率间隔由 f_s/N 增至 f_s/L，故 Welch 谱仅承担稳健性复核，全长 FFT 仍用于报告最终频率。

最终故障频率估计为功率谱达到最大值的位置：

$$\hat{f}_0 = \underset{f\in(0,f_s/2)}{\arg\max}\ P(f), \quad P \in \{P_{\mathrm{FFT}}, P_W\}. \tag{6}$$

为控制在 $0 \sim 20\,\mathrm{Hz}$ 全频带搜索造成的多重比较效应，本文以最大谱线而非单个预定频点构造检验。将系统允许的总虚警率记为 P_{FA}，Bonferroni 校正把每条谱线的显著性水平分配为 $\alpha = P_{FA}/B$。白噪声条件下，以噪声均值归一化的单频点周期图近似服从单位均值指数分布，故 $\Pr\{Y > \eta\} = e^{-\eta}$，从而得到阈值

$$\eta = -\ln\left(\frac{P_{FA}}{B}\right). \tag{7}$$

由并集上界可知，$\Pr\{\max_b Y_b > \eta\} \le B\alpha = P_{FA}$，因此该校正不要求各谱线严格独立，且比逐频点判断更能避免“搜索频点越多、偶然峰越多”的问题。本题在 $0 \sim 20\,\mathrm{Hz}$ 内搜索 $B = 8000$ 个正频率谱线，取总虚警率 $P_{FA} = 0.01$ 时阈值约为噪声均值的 13.59 倍，而观测峰值比约为 1072，故结论不依赖主观画线。该阈值仍依赖单频点指数分布近似；若残差呈显著有色特征，应改用预白化或残差 Bootstrap。Welch 采用 Hann 窗、$L = 8192$、50% 重叠，频率分辨率为 $0.012207\,\mathrm{Hz}$，用于稳健复核而非替代全长 FFT 的精定位。

本模型输出单源特征频率及噪声基底，前者直接进入问题二的谐波参数估计，二者共同为后续检测阈值和仿真噪声尺度提供依据。

6.2 问题二：单源故障波形恢复模型

在问题一得到 f_0 后，正弦信号的未知参数只剩振幅和相位。利用三角恒等式将非线性参数估计转化为线性回归：

$$s(t) = A\sin(2\pi f_0 t + \varphi_0) = c_s \sin(2\pi f_0 t) + c_c \cos(2\pi f_0 t), \tag{8}$$

其中

$$c_s = A\cos\varphi_0, \quad c_c = A\sin\varphi_0.$$

对所有采样点构造设计矩阵：

$$\Phi = \begin{bmatrix} \sin(2\pi f_0 t_1) & \cos(2\pi f_0 t_1) \\ \vdots & \vdots \\ \sin(2\pi f_0 t_N) & \cos(2\pi f_0 t_N) \end{bmatrix}, \quad \theta = \begin{bmatrix} c_s \\ c_c \end{bmatrix}. \tag{9}$$

最小二乘估计解析解为[9]：

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T x. \tag{10}$$

由 $\hat{c}_s$、$\hat{c}_c$ 还原振幅与初相：

$$\hat{A} = \sqrt{\hat{c}_s^2 + \hat{c}_c^2}, \quad \hat{\varphi}_0 = \operatorname{atan2}(\hat{c}_c, \hat{c}_s). \tag{11}$$

由于附件未提供无噪声真值，本文不能直接计算 $\hat{s}$ 与真实 s 的恢复误差，而以残差均方根和分段参数稳定性作为间接评价，并用已知真值的残差重采样仿真检验恢复算法本身。残差均方根为

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} \left(x[k] - \hat{s}[k]\right)^2}. \tag{12}$$

本模型输出单源振幅、初相及恢复波形，并通过附件残差检验和有真值仿真分别评价数据拟合质量与算法恢复误差。

6.3 问题三：多故障源自动分离模型

多源场景下信号可表示为多个弱周期分量叠加：

$$x(t) = \sum_{i=1}^{M} A_i \sin(2\pi f_i t + \varphi_i) + n(t). \tag{13}$$

由于 M 未知，本文先通过宽松阈值从 FFT 功率谱中提取候选峰，再对不同阶数 M 构造联合最小二乘模型，利用 BIC 抑制过拟合[10]：

$$\mathrm{BIC}(M) = N \ln\left(\frac{RSS_M}{N}\right) + k_M \ln N. \tag{14}$$

其中 k_M 为参数个数。虽然候选峰在选阶阶段暂时固定，但频率由同一数据筛选，仍会消耗模型自由度，为避免低估复杂度，本文保守取 $k_M = 3M$，分别对应频率、正弦系数和余弦系数。确定 M^* 后，再连续优化频率并联合更新全部参数。

多频率联合最小二乘模型为：

$$x(t) = \sum_{i=1}^{M} [\alpha_i \sin(2\pi f_i t) + \beta_i \cos(2\pi f_i t)] + n(t). \tag{15}$$

本模型输出周期分量数及各分量的频率、振幅和初相，为问题四的多频率检测器、源强输入和最弱源鲁棒准则提供参数。

6.4 问题四：传感器融合检测与布局优化模型

问题四需要把信号估计结果转化为空间布设方案。设第 i 个传感器到第 j 个故障源的距离为 d_{ij}，g_{ij} 表示传播路径和安装方式造成的灵敏度差异，接收振幅为

$$A_{ij} = A_j g_{ij} \exp(-\alpha d_{ij}), \tag{16}$$

其中 A_j 直接采用问题三估计的各源振幅。允许噪声随测点变化，单样本信噪比为

$$\rho_{ij} = \frac{A_{ij}^2}{2\sigma_i^2}. \tag{17}$$

对每个已知故障频率，用正、余弦两个正交分量构造非相干能量统计量。选择 q 个传感器、每个检测窗含 N_o 个样本时，零假设和备择假设下分别有

$$Z_j \mid H_0 \sim \chi^2_{2q}, \qquad Z_j \mid H_1 \sim \chi'^2_{2q}(\lambda_j), qquad \lambda_j = N_o \sum_{i \in \mathcal{S}} \rho_{ij}. \tag{18}$$

同时监测 $J = 4$ 个故障频率时，先给定系统总虚警率 P_{FA}^{sys}。为避免依赖不同频率统计量相互独立这一理想条件，本文采用 Bonferroni 分配

$$P_{FA}^{(1)} = \frac{P_{FA}^{\mathrm{sys}}}{J}, \qquad P\left(\bigcup_{j=1}^{J} \{Z_j > \gamma_q\} \,\middle|\, H_0\right) \le \sum_{j=1}^{J} P_{FA}^{(1)} = P_{FA}^{\mathrm{sys}}. \tag{19}$$

因此单频阈值与检测概率为

$$\gamma_q = F^{-1}_{\chi^2_{2q}}\left(1 - P^{(1)}_{FA}\right), \qquad P^{(j)}_D = 1 - F_{\chi'^2_{2q}(\lambda_j)}(\gamma_q). \tag{20}$$

该统计量是噪声归一化的非相干能量融合，而非需要已知跨测点相位的相干最大比合并。独立传感器的信息贡献按信噪比相加：

$$\rho^{(j)}_{\mathrm{eq}} = \sum_{i\in\mathcal{S}} \rho_{ij}. \tag{21}$$

在候选安装点集合 Ω 上，最多三个传感器的鲁棒布局模型为

$$\max_{\mathcal{S}\subseteq\Omega,\,1\le|\mathcal{S}|\le 3} \min_j P^{(j)}_D(\mathcal{S}). \tag{22}$$

题目要求兼顾检测与误报。本文采用 Neyman–Pearson 固定虚警率思想[11]：先约束系统总 P^{sys}_{FA}，再最大化最差源的 P_D。由于未知相位使备择假设成为复合假设，本文不宣称该能量统计量在所有检验中一致最优。因题目没有说明四个频率与空间源位置的一一对应关系，算例按问题三最弱振幅、$\alpha = 0.8$、噪声尺度 1.2 及 $g_{ij} = 0.85$ 的不利组合进行保守设计，再在名义参数和扰动参数下评估同一布局。

本模型输出不超过三个测点的区域级布局及其检测概率、虚警率和报警时间；获得真实图纸与标定数据后，只需替换传播矩阵和噪声协方差即可重新求得安装坐标。

七、 模型结果与分析

7.1 问题一结果：单源频率检测

FFT 精细谱定位得到故障峰

$$\hat{f}_0 = 2.0000\,\mathrm{Hz},$$

谱峰与噪声中位基底之比约为 1782；Welch 估计在 $f \approx 2.0020\,\mathrm{Hz}$ 处给出一致峰值，差异仅为一个 Welch 频率分辨单元量级。如图 2 所示，时域波形难以直接辨识周期性，而两种频谱均在 2 Hz 附近形成稳定峰值。

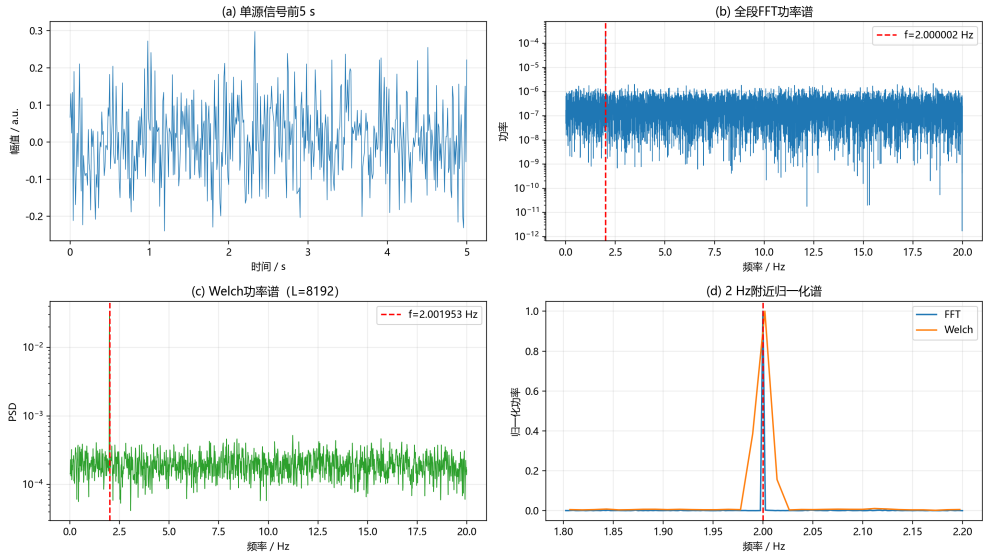


图 **2**　单源故障信号时频分析

表 **3**　三种频谱分析方法定量对比

分析方法	Δf/Hz	分段数	峰值/谱中位基底	噪声基底 CV	全频带虚警控制
STFT $L=256$	0.390625	311	6.21	–	否
全段 FFT $L=N$	0.002500	1	1782.04	1.004	否
Welch $L=8192$	0.012207	8	176.66	0.361	联合检验后是

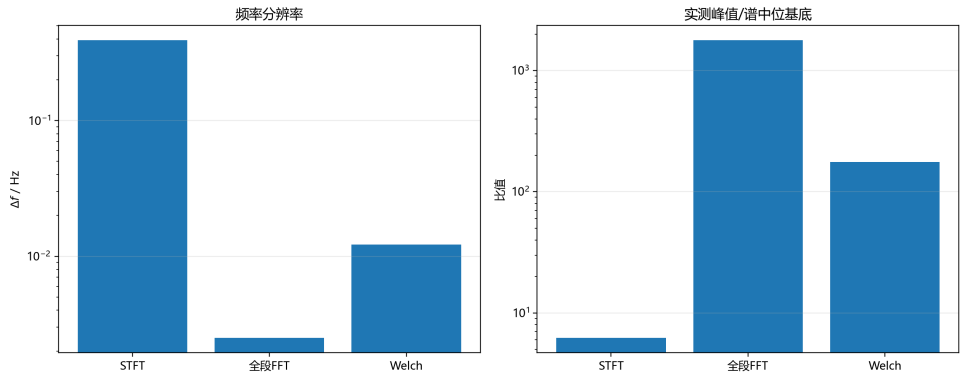


图 **3**　频谱分析方法性能对比

图 3 与表 3 均直接由附件数据计算，不再混用不同自由度下的理论检测概率。STFT 保留时间定位能力但频率分辨率最低；全段 FFT 适合精确定位但单次周期图波动较大；Welch 把噪声谱基底离散系数由 1.004 降至 0.361，降低约 64.1%，且八个输入分段均在 2 Hz 附近检出主峰。因而本文以全段 FFT 定位、

Welch 复核，再联合最大谱线检验控制全频带虚警；相较只查看传统 FFT 谱峰，该流程同时给出跨时段稳定性和统计显著性证据。

7.2 问题二结果：波形恢复与误差评价

连续频率精化与最小二乘联合求解得到

$$\hat{A} = 0.0352, \quad \hat{\varphi}_0 = 1.574\,\text{rad}.$$

恢复信号为：

$$\hat{s}(t) = 0.0352\sin(2\pi \times 2.0000t + 1.574). \tag{23}$$

残差 RMSE $= 0.1000$，最大 1–20 阶绝对自相关为 0.0094，Jarque–Bera 检验 $p = 0.669$，支持高斯白噪声近似。分别使用前、后 200 s 数据估计得到振幅 0.03585 和 0.03453，相差 3.7%，说明周期参数在观测期内稳定。由于真实无噪声波形未知，以上指标评价的是模型残差和参数稳定性，不能解释为 $\hat{s}$ 对真实 s 的直接 RMSE。如图 4 所示，恢复分量具有稳定周期，残差未保留明显同频结构。

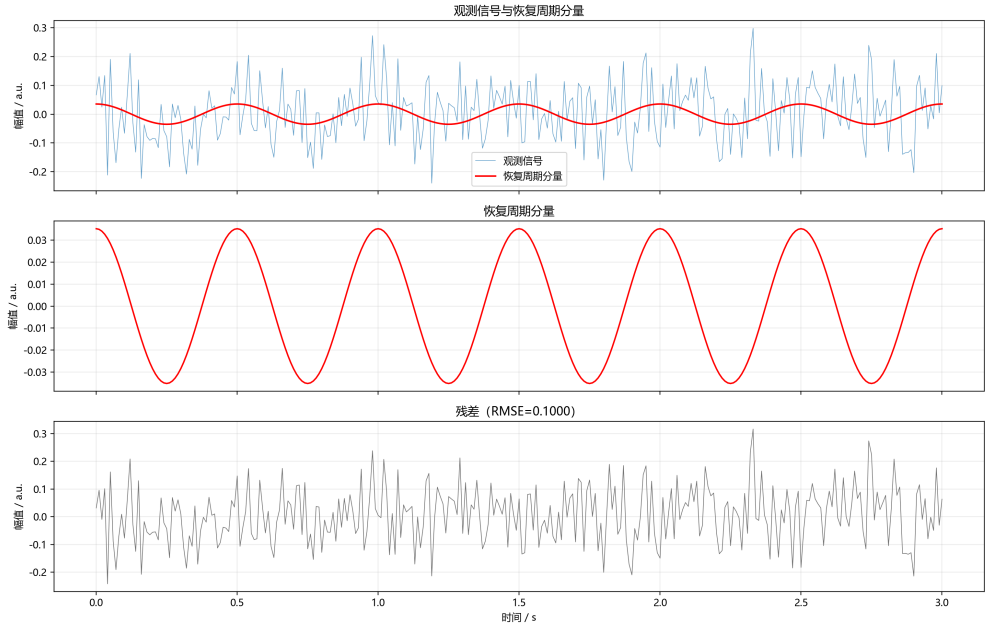


图 **4**　单源故障信号恢复与残差

表 **4**　单源故障波形恢复结果

指标	估计值	解释
故障频率 f_0	2.0000 Hz	由 FFT 精细定位并经 Welch 验证
振幅 A	0.0352 a.u.	约为残差标准差的 35%
初相 φ_0	1.574 rad	约为 90.2°，接近余弦形态
残差 RMSE	0.1000	表征未解释部分，不是真实波形误差
A 的条件 95% CI	$[0.0338, 0.0366]$ a.u.	条件于已估频率及白噪声近似
φ_0 的条件 95% CI	$[1.534, 1.613]$ rad	不包含频率搜索的不确定性

表 4 中的区间是条件于已估频率和高斯白噪声近似的参数区间；其用途是量化回归系数不确定性，而不是覆盖频率搜索及模型失配的完整置信区间。

为补足附件缺少无噪声真值的限制，本文以式 (23) 为已知真值，从附件残差中有放回抽样构造同尺度噪声，重复 500 次“FFT 连续峰值精化–最小二乘恢复”。表 5 给出各次误差的中位数及经验 95% 区间；该实验验证算法在已知真值条件下的恢复能力，而不把合成真值冒充附件真实故障波形。

表 **5**　单源波形有真值仿真误差（**500** 次）

指标	中位数	经验 95% 区间
频率绝对误差 / Hz	5.17×10^{-5}	$[4.97, 5.38] \times 10^{-5}$
振幅相对误差	1.38%	0.051%–4.41%
相位绝对误差 / rad	0.0653	0.0257–0.1041
波形 NRMSE	4.47%	3.76%–6.44%

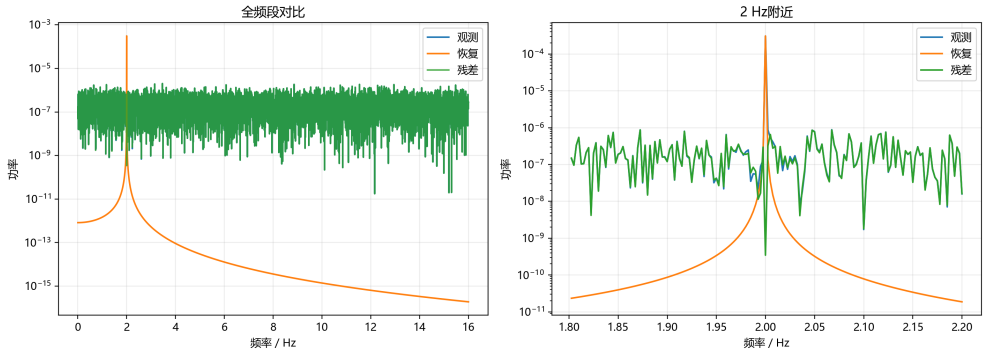


图 **5**　信号恢复结果的频域对比

图 5 进一步表明，恢复分量的能量集中于 2 Hz，残差在该频率附近未出现同量级窄带峰。

7.3 问题三结果：多故障源分离

对多源信号应用宽松寻峰与 BIC 自动选阶，BIC 在 $M = 4$ 处取得最小值；$\mathrm{BIC}(5) - \mathrm{BIC}(4) = 13.36$，表明增加第五个分量所得残差改善不足以抵消复杂度惩罚。在固定候选峰集合下进行 500 次条件残差 Bootstrap，四阶模型被选择 500 次，选阶比例为 100%；该结果证明选阶对残差扰动稳定，但不覆盖候选峰提取规则改变带来的不确定性。

表 **6**　多故障源联合最小二乘估计结果

故障频率 f_i/Hz	振幅 A_i	初相 φ_i/rad	初相 /°	工程解释
4.0000	0.0191	1.372	78.6	低频分量
7.9999	0.0249	-2.989	-171.3	第二分量
13.0001	0.0105	-2.157	-123.6	最弱分量
14.0000	0.0401	0.112	6.4	最强分量

从表 6 可见，数据支持四阶周期模型，但这并不等同于已知真实物理源数。图 6 给出了 FFT 候选峰及 BIC 选阶曲线。

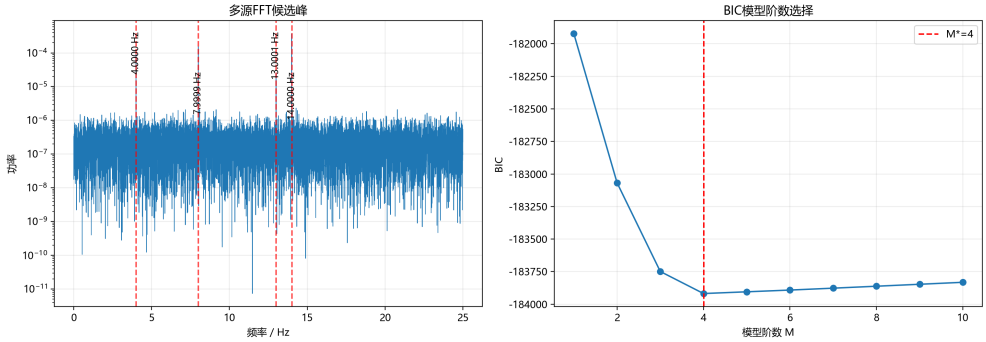


图 **6**　多源频谱寻峰与 BIC 选阶

在多源时域分离中，采用连续频率联合最小二乘重建四个周期分量。如图 7 所示，本文分离的是可辨识的周期分量，不把它们过度解释为已完成物理源定位。

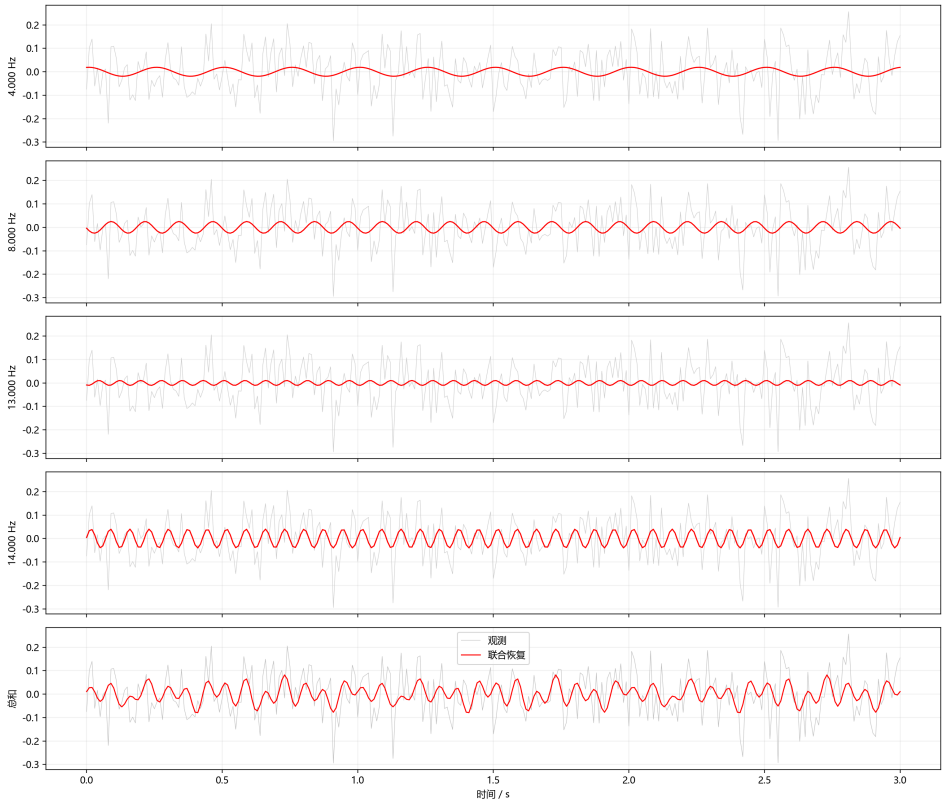


图 **7**　多故障源时域波形分离

多频率联合恢复后的残差 RMSE 为 0.0999。图 8 所示的 500 次蒙特卡洛实验表明：间隔 0.0025 Hz 时成功率为 6.6%，Wilson 95% 区间为 4.74%–9.12%；间隔达到 0.005 Hz 后 500 次均成功，对应成功概率的 Wilson 95% 区间为 99.24%–100%，故本文数据条件下的经验分辨界约为 0.005 Hz，而实际最小间距为 1 Hz。该结论是给定振幅、相位分布和噪声条件下的经验界，不宣称为普适理论极限。

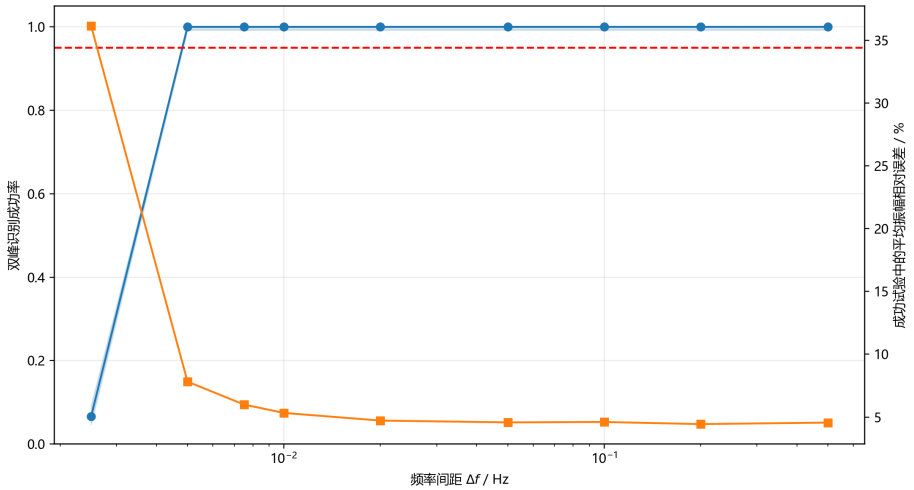


图 **8**　相邻双频分辨实验

7.4 问题四结果：传感器布局优化

在代表性算例的 0.25 m 网格上穷举 187460 种三传感器组合。名义评估取 $\alpha = 0.5$、基准噪声 0.1000 并设中心附近噪声升高 35%、$g_{ij} = 1$；布局设计则采用最弱振幅 0.0105、$\alpha = 0.8$、噪声尺度 1.2 和 $g_{ij} = 0.85$ 的不利组合。图 10 和表 7 给出该代表性场景的鲁棒布点。真实设备应用时须将算例几何和参数替换为图纸及标定数据后重新运行算法。

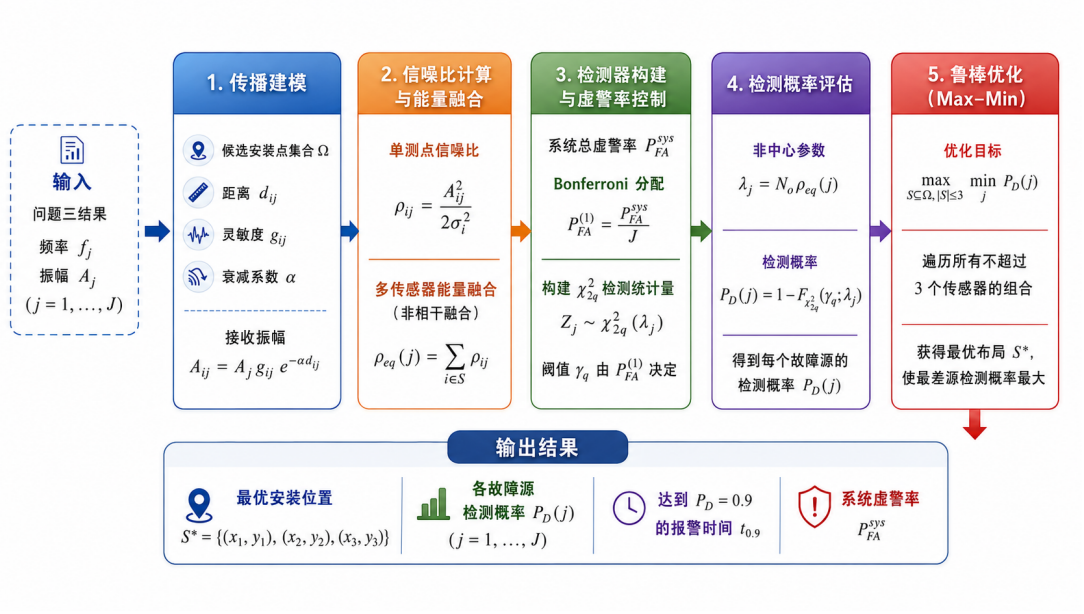


图 **9**　问题四传感器布局优化流程图

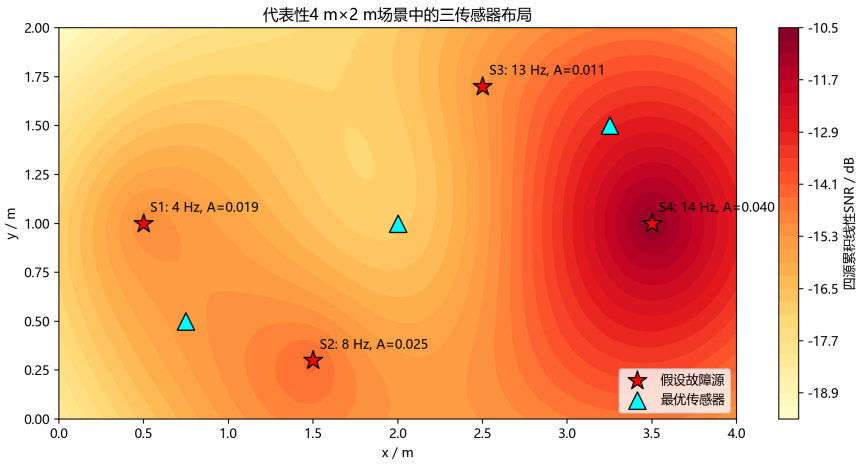


图 **10**　代表性算例的鲁棒布局

表 **7**　代表性算例的鲁棒三传感器布局

编号	安装坐标 / m	设计作用	备注
T1	$(0.75, 0.50)$	覆盖左侧潜在源	代表性算例坐标
T2	$(2.00, 1.00)$	覆盖中部潜在源	代表性算例坐标
T3	$(3.25, 1.50)$	覆盖右侧潜在源	代表性算例坐标

表 **8**　传感器数量对检测性能的影响

方案	最差源 SNR	最差 P_D（4 s）	达到 $P_D = 0.9$ 的时间
鲁棒最优 1 传感器	−30.151 dB	0.0060	220.7 s
鲁棒最优 2 传感器	−24.930 dB	0.0121	77.8 s
鲁棒最优 3 传感器	−23.661 dB	**0.0132**	**64.4 s**

表 8 汇总不同传感器数量下的名义性能。图 11 分别给出以系统总虚警率为横轴的 ROC 曲线，以及故障功率尺度变化下的最差检测概率。三传感器方案的排序保持领先，但短时弱源检测仍然困难，不能把“相对改善”夸写成“已高概率检出”。

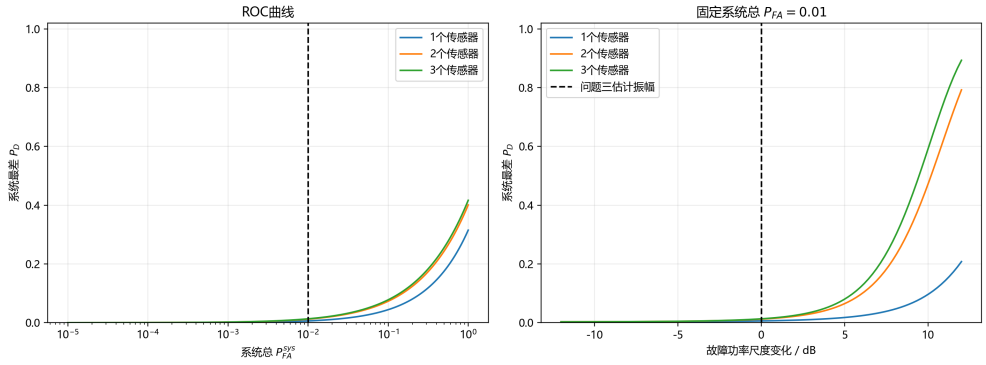


图 **11**　系统 **ROC** 与故障功率敏感性

结果表明，三传感器通过空间分集改善最弱覆盖：系统总 $P_{FA}^{\mathrm{sys}} = 0.01$、4 s 观测时，最差 P_D 由 0.0060 提高至 0.0132；更有工程意义的是达到 $P_D = 0.9$ 的观测时间由 220.7 s 缩短至 64.4 s。对四个振幅与四个位置的全部排列复核后，三点布局最差 P_D 位于 0.0120–0.0132，说明保守布局对未知对应关系不敏感。

工程落地流程与有效性边界

题目没有提供足以唯一确定物理坐标的设备图纸和多测点标定数据，因此本文不能把表 7 的算例坐标解释为真实设备最终安装点。实际应用应按以下流程替换算例输入，而无须改变优化框架：

（1） 根据设备图纸、安装可达性和安全距离建立真实候选点集合 Ω_{real}，并标注轴承座、齿轮箱壳体等关键区域；

（2） 在停机敲击或可控激励下测得各候选点到各潜在源的频率响应 $H_{ij}(f_j)$，以实测 $|H_{ij}|$ 替代指数衰减中的 $g_{ij}e^{-\alpha d_{ij}}$；

（3） 在健康工况下同步采集多测点噪声，估计位置相关方差及跨测点协方差，并在相关噪声下采用协方差预白化；

（4） 将前三问得到的 A_j, f_j、系统总虚警率及上述标定量代入式 (20)–(22)，重新搜索不超过三个安装点；

（5） 用未参与标定的多测点数据验证系统总虚警率和最差检测概率，若理论分布失配，则以健康残差 Bootstrap 标定联合阈值。

问题四最终布局方案：对具有左右轴承支承和中部齿轮箱的典型旋转机械，三个传感器采用“左轴承座–齿轮箱壳体–右轴承座”的区域级分散布局；若设备结构不同，则将三者替换为沿主要传播路径分散的三个关键承载部位。每个区域优先选择背景噪声低、安装刚度高且对最弱故障频率响应最大的候选点。获得现场标定数据后，最终坐标唯一按

$$\mathcal{S}^* = \arg \max_{\mathcal{S}\subseteq\Omega_{\text{real}},\,|\mathcal{S}|\le 3} \min_j P_D^{(j)}(\mathcal{S})$$

确定。若某区域候选点的灵敏度或噪声不满足要求，算法允许将该点移至其他关键部位，故区域级建议不替代数据驱动的最终优化。

代表性算例验证的是布局优化与统计检测流程的可执行性，不构成对真实设备传播模型及最终安装坐标的验证。

八、 模型检验

8.1 衰减系数灵敏度

由于衰减系数与噪声场均需实测标定，本文固定表 7 的鲁棒布局，对 α 和噪声尺度同时扰动，而不是在知道扰动结果后重新选点。表 9 给出噪声尺度为 1 时的结果。

表 **9**　衰减系数灵敏度

衰减系数 α	最差源 SNR/dB	最差 P_D
0.3	-21.845	0.0232
0.4	-22.775	0.0171
0.5	-23.661	0.0132
0.6	-24.508	0.0105
0.8	-26.117	0.0073

衰减增强或噪声上升均会降低绝对检测概率，这说明布局效果依赖工程标定，不能脱离参数报告单一“最优坐标”。另令 g_{ij} 在 $[0.85, 1.15]$ 独立扰动 5000 次，最差 P_D 的 5%、50%、95% 分位数为 0.0106、0.0132、0.0165。

8.2 理论分布与蒙特卡洛核验

系统总虚警率 0.01 经 Bonferroni 分配后，单频率虚警率为 0.0025；若额外假设四个频率检验独立，Šidák 值为 0.002509。两种口径下三传感器 4 s 最差 P_D 分别为 0.01315 和 0.01319，相差不足 0.3%，说明保守控制几乎不牺牲性能。在正交匹配滤波输出层进行 10^5 次蒙特卡洛校准，四频率同时监测的经验系统虚警率为 0.00995，理论检测概率与模拟值一致。图 12 验证的是阈值和检测器数值实现；由于缺少真实多测点数据，它不能验证指数传播模型本身。

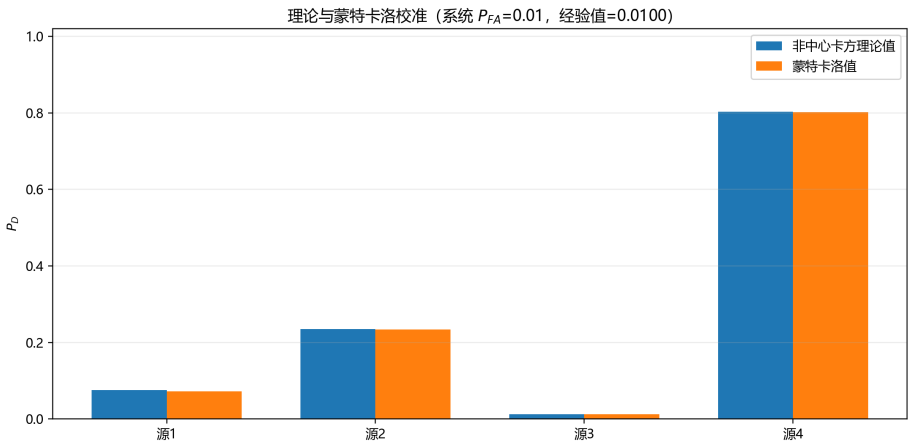


图 **12**　理论值与蒙特卡洛核验

8.3 积累时长影响

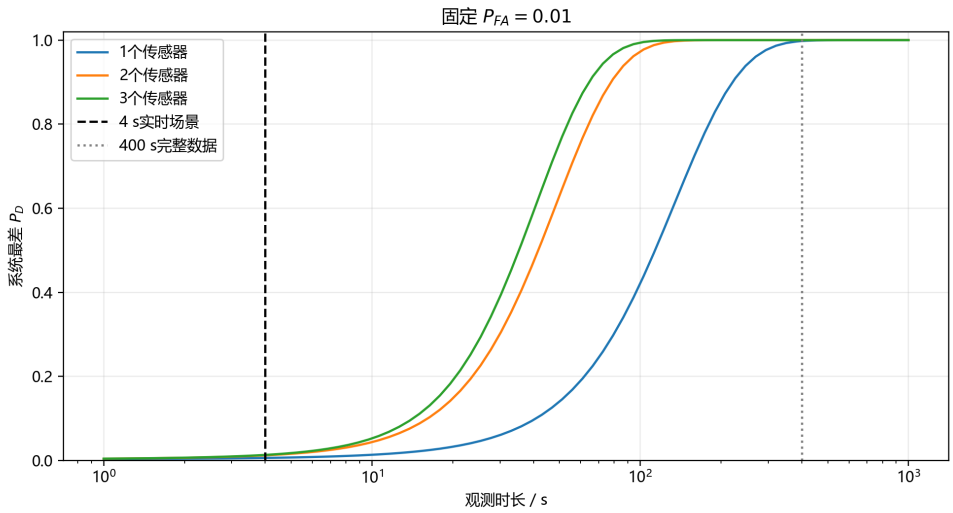


图 **13**　积累时长对检测概率的影响

图 13 表明观测时长越长，三种方案均逐渐接近完全检测；三传感器达到 $P_D = 0.9$ 约需 64.4 s，二传感器需 77.8 s，单传感器需 220.7 s。若工程要求数秒内报警，则需提高传感器灵敏度、降低噪声或调整系统虚警约束，不能仅靠当前布局满足。

8.4 虚警率约束灵敏度

进一步改变系统总虚警率约束 P_{FA}^{sys}，每个水平都按式 (19) 分配到四个频率，再比较不同方案的最差检测概率，结果见表 10。

表 **10**　不同虚警率约束下最差故障源检测概率

系统 P_{FA}^{sys}	最优 1 传感器 P_D	最优 2 传感器 P_D	最优 3 传感器 P_D
0.001	0.0008	0.0019	0.0021
0.010	0.0060	0.0121	**0.0132**
0.050	0.0243	0.0428	0.0459

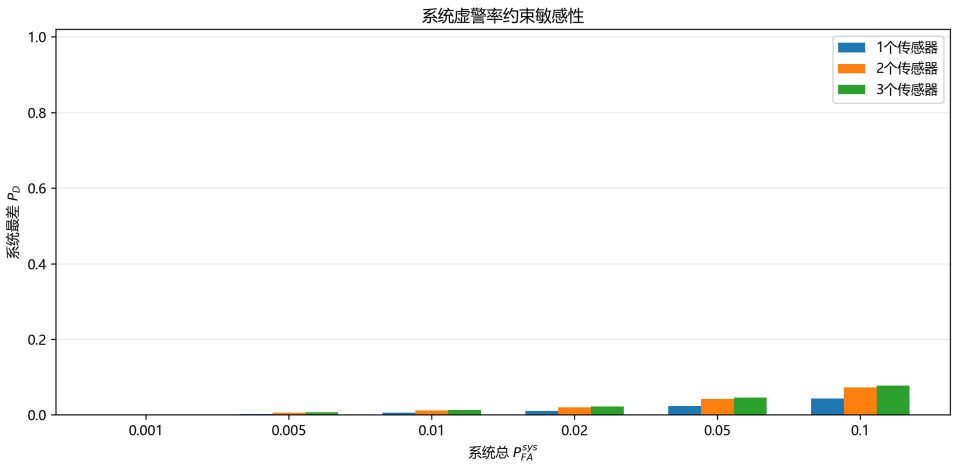


图 **14**　虚警率约束下的检测性能

图 14 表明，在六个系统总虚警率水平下三传感器均优于单传感器；同时，绝对 P_D 会随 P_{FA}^{sys} 明显变化。因此可靠表述应为“方案排序稳定”，而不是“检测概率对虚警率不敏感”。

8.5 消融实验与可复现性检查

表 11 从方法链条角度说明各模块的必要性；该表属于结构性分析，不替代前述数值敏感性和蒙特卡洛校准。

表 **11**　模型关键模块消融分析

消融项	去除后的影响	结论
去除 Welch 验证	仅凭单段 FFT，噪声基底起伏增大，峰值稳定性证明不足	Welch 提供稳健性证据
去除 BIC 选阶	需要人工指定 M，容易漏检弱峰或过拟合噪声峰	BIC 使多源分离自动化
去除联合 LS	各频率分量逐个拟合会产生相互泄漏	联合估计能同时校正各分量
去除非相干能量融合	传感器只能局部检测，无法提升全局最差 P_D	融合是提升系统级性能的关键
去除 max-min 目标	可能使强源检测过好、弱源漏检	max-min 保证最差故障源可靠性

九、 模型评价与改进

模型优点：四问由检测、恢复、分离到布局递进，谱估计、正交投影、BIC、Bonferroni 控制与 max-min 优化相互衔接；频率、振幅、区间、残差、P_D 和灵敏度均有定量结果。真实图纸、传播标定和噪声测量可直接替换算例输入，算法具有可核验性与可迁移性。

模型不足：非平稳冲击、转速谐波和测点相关噪声仍需现场检验；指数衰减未描述反射、多路径及局部刚度变化；恒速假设不适用于明显变速工况；候选点或传感器数量很大时，穷举计算量迅速增加。

推广改进：变速工况可引入阶次谱或同步重采样，未知频率检测可采用 GLRT 与 Bootstrap；系统层面可用实测频率响应替代指数衰减，并把成本、安装可达性和冗余容错纳入多目标布局。

十、 结论

本文对原始数据、统计量尺度和问题四题设条件进行了统一核验。单源频率为 2.0000 Hz，恢复振幅与初相为 0.0352 和 1.574 rad；500 次有真值仿真的波形 NRMSE 中位数为 4.47%。多源模型由 BIC 选择四阶，$\Delta\mathrm{BIC}_{5-4} = 13.36$，连续精化频率为 4.0000、7.9999、13.0001、14.0000 Hz，残差 RMSE 为 0.0999。

问题四在系统总虚警率 0.01 下给出 max–min 三点布局：典型设备布于左右轴承座与中部齿轮箱，其他结构沿传播路径分散布于关键承载部位。代表性算例中，三传感器使达到 $P_D = 0.9$ 的时间由 220.7 s 降至 64.4 s。

参考文献

[1] LV Y, YANG D, YUAN R, et al. A novel multivariate signal processing-based fault diagnosis approach of rotating machinery under various operating conditions[J]. Measurement Science and Technology, 2022, 33(7): 075106. DOI: 10.1088/1361-6501/ac60d5.

[2] AN H, YOUN B D, KIM H S. Optimal sensor placement considering both sensor faults under uncertainty and sensor clustering for vibration-based damage detection[J]. Structural and Multidisciplinary Optimization, 2022, 65(3): 102. DOI: 10.1007/s00158-021-03159-9.

[3] HASSANI S, DACKERMANN U. A systematic review of optimization algorithms for structural health monitoring and optimal sensor placement[J]. Sensors, 2023, 23(6): 3293. DOI: 10.3390/s23063293.

[4] XIAO X, LI C, HE H, et al. Rotating machinery fault diagnosis method based on multi-level fusion framework of multi-sensor information[J]. Information Fusion, 2025, 113: 102621. DOI: 10.1016/j.inffus.2024.102621.

[5] DeepSeek, DeepSeek-V3.1, 深度求索（DeepSeek）, 2026.06.23.

[6] PROAKIS J G, MANOLAKIS D G. Digital Signal Processing[M]. 4th ed. Upper Saddle River: Pearson, 2007.

[7] BENDAT J S, PIERSOL A G. Random Data: Analysis and Measurement Procedures[M]. 4th ed. Hoboken: Wiley, 2010.

[8] WELCH P D. The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms[J]. IEEE Transactions on Audio and Electroacoustics, 1967, 15(2): 70–73. DOI: 10.1109/TAU.1967.1161901.

[9] KAY S M. Fundamentals of Statistical Signal Processing, Volume I: Estimation Theory[M]. Englewood Cliffs: Prentice Hall, 1993.

[10] SCHWARZ G. Estimating the dimension of a model[J]. The Annals of Statistics, 1978, 6(2): 461–464. DOI: 10.1214/aos/1176344136.

[11] NEYMAN J, PEARSON E S. On the problem of the most efficient tests of statistical hypotheses[J]. Philosophical Transactions of the Royal Society A, 1933, 231: 289–337. DOI: 10.1098/rsta.1933.0009.

附录：支撑材料与完整程序

附录 A：支撑材料清单

根据竞赛论文格式规范，附录列出支撑论文结果的文件。原始试验代码不作为论文结果依据，也不在附录展示；表 12 中的最终程序是唯一用于生成本文数值、表格和图件的版本。

表 **12**　支撑材料文件清单

序号	文件或目录	说明
1	`code/final_analysis.py`	最终完整程序；统一完成问题一至问题四并生成全部论文结果
2	`code/requirements.txt`	Python 依赖及经复现实验验证的版本
3	`generated/results.json`	论文关键数值的机器可读汇总
4	`generated/tables/`	方法比较、BIC、分辨实验、布局及灵敏度结果
5	`generated/figures/`	由最终程序生成的图 1–12

附录 B：运行环境与复现方法

运行环境为 Windows 11、Python 3.13.13；主要依赖为 NumPy 2.4.4、Pandas 3.0.2、SciPy 1.17.1、Matplotlib 3.10.8 和 openpyxl 3.1.5。在论文根目录执行

```
python -m pip install -r code/requirements.txt
python code/final_analysis.py
```

即可读取题目附件 `A 题：机械设备故障检测/data.xlsx`，并在 `generated/` 下重新生成全部输出。随机实验采用固定种子 `20260622`，以保证结果可复现。

附录 C：最终完整源代码

Listing 1: 问题一至问题四统一求解程序

```
"""A题机械设备故障检测：论文最终可复现程序。

运行方式：在论文工作目录执行 ``python code/final_analysis.py``。
程序直接读取题目附件 data.xlsx，统一完成问题一至问题四的计算，
并在 generated 目录输出全部数值结果、数据表及论文图1—12。
"""

from __future__ import annotations

import itertools
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import minimize, minimize_scalar
from scipy.signal import find_peaks, spectrogram, welch
from scipy.stats import chi2, jarque_bera, ncx2
```

```python
ROOT = Path(__file__).resolve().parents[1]
DATA_FILE = ROOT / "A题：机械设备故障检测" / "data.xlsx"
OUT = ROOT / "generated"
FIG = OUT / "figures"
TABLE = OUT / "tables"
OUT.mkdir(exist_ok=True)
FIG.mkdir(exist_ok=True)
TABLE.mkdir(exist_ok=True)

FS = 100.0
PFA = 0.01 # 问题一全频带虚警率；问题四中表示四频率联合监测的系统总虚警率
RNG = np.random.default_rng(20260622)

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False


def load_sheet(index: int):
    df = pd.read_excel(DATA_FILE, sheet_name=index)
    t = df.iloc[:, 0].to_numpy(float)
    x = df.iloc[:, 1].to_numpy(float)
    return t, x


def fft_spectrum(t, x):
    dt = float(np.median(np.diff(t)))
    y = x - np.mean(x)
    f = np.fft.rfftfreq(len(y), dt)
    p = np.abs(np.fft.rfft(y) / len(y)) ** 2
    return f, p


def ls_at_frequencies(t, x, frequencies):
    cols = []
    for f in frequencies:
        cols.extend([np.sin(2 * np.pi * f * t), np.cos(2 * np.pi * f * t)])
    phi = np.column_stack(cols)
    coef, *_ = np.linalg.lstsq(phi, x, rcond=None)
    fitted = phi @ coef
    residual = x - fitted
    params = []
    for i, f in enumerate(frequencies):
        a, b = coef[2 * i : 2 * i + 2]
        params.append(
            {
                "frequency": float(f),
                "sin_coef": float(a),
                "cos_coef": float(b),
                "amplitude": float(np.hypot(a, b)),
                "phase": float(np.arctan2(b, a)),
            }
        )
    return params, fitted, residual, float(np.sum(residual**2))


def refine_single_frequency(t, x, f_init, half_width=0.01):
    def objective(f):
        return ls_at_frequencies(t, x, [float(f)])[3]

    result = minimize_scalar(
        objective,
        bounds=(f_init - half_width, f_init + half_width),
        method="bounded",
        options={"xatol": 1e-12},
    )
    return float(result.x)


def refine_multiple_frequencies(t, x, initial, half_width=0.01):
    initial = np.sort(np.asarray(initial, float))
    bounds = [(f - half_width, f + half_width) for f in initial]

    def objective(freqs):
        if np.any(np.diff(np.sort(freqs)) <= 0):
            return 1e12
        return ls_at_frequencies(t, x, np.sort(freqs))[3]

    result = minimize(objective, initial, method="L-BFGS-B", bounds=bounds)
    return np.sort(result.x)
```

```python
def q1_q2(t, x):
    f, p = fft_spectrum(t, x)
    peak = np.argmax(p[1:]) + 1
    f_fft = float(f[peak])
    f_refined = refine_single_frequency(t, x, f_fft)
    params, fitted, residual, _ = ls_at_frequencies(t, x, [f_refined])
    par = params[0]

    fw, pw = welch(
        x - x.mean(), fs=FS, window="hann", nperseg=8192, noverlap=4096
    )
    wp = np.argmax(pw[1:]) + 1

    # Conditional covariance at the refined frequency.
    phi = np.column_stack(
        [np.sin(2 * np.pi * f_refined * t), np.cos(2 * np.pi * f_refined * t)]
    )
    sigma2 = np.sum(residual**2) / (len(x) - 2)
    cov = sigma2 * np.linalg.inv(phi.T @ phi)
    draws = RNG.multivariate_normal(
        [par["sin_coef"], par["cos_coef"]], cov, size=20000
    )
    amp_draw = np.hypot(draws[:, 0], draws[:, 1])
    phase_draw = np.arctan2(draws[:, 1], draws[:, 0])

    # Residual diagnostics and split-sample stability.
    residual_centered = residual - residual.mean()
    residual_var = float(np.dot(residual_centered, residual_centered))
    residual_acf = [
        float(np.dot(residual_centered[:-lag], residual_centered[lag:]) / residual_var)
        for lag in range(1, 21)
    ]
    jb = jarque_bera(residual_centered)
    half = len(x) // 2
    first_params, *_ = ls_at_frequencies(t[:half], x[:half], [f_refined])
    second_params, *_ = ls_at_frequencies(t[half:], x[half:], [f_refined])

    # Global FFT maximum threshold over 0--20 Hz under exponential-bin approximation.
    bins = int(np.sum((f > 0) & (f <= 20)))
    global_threshold_over_mean = -np.log(1 - (1 - PFA) ** (1 / bins))
    observed_over_mean = float(p[peak] / np.mean(p[(f > 0) & (f <= 20)]))

    # Cross-segment evidence and observed noise-floor fluctuation for the
    # comparison with a single traditional periodogram. The eight Hann
    # segments are exactly those averaged by the Welch estimate.
    fseg, tseg, pseg = spectrogram(
        x - x.mean(), fs=FS, window="hann", nperseg=8192, noverlap=4096,
        mode="psd"
    )
    search_seg = (fseg > 0) & (fseg <= 20)
    segment_peaks = fseg[search_seg][np.argmax(pseg[search_seg, :], axis=0)]
    segment_detection_rate = float(
        np.mean(np.abs(segment_peaks - f_refined) <= FS / 8192)
    )
    noise_fft = (f > 0) & (f <= 20) & (np.abs(f - f_refined) > 0.2)
    noise_welch = (fw > 0) & (fw <= 20) & (np.abs(fw - f_refined) > 0.2)
    fft_noise_cv = float(np.std(p[noise_fft]) / np.mean(p[noise_fft]))
    welch_noise_cv = float(np.std(pw[noise_welch]) / np.mean(pw[noise_welch]))

    result = {
        "N": len(x),
        "fs": float(1 / np.median(np.diff(t))),
        "duration": float(t[-1] - t[0]),
        "mean": float(np.mean(x)),
        "std": float(np.std(x)),
        "f_fft_bin": f_fft,
        "f_refined": f_refined,
        "fft_peak_over_median": float(p[peak] / np.median(p[1:])),
        "welch_L": 8192,
        "welch_peak": float(fw[wp]),
        "welch_peak_over_median": float(pw[wp] / np.median(pw[1:])),
        "global_bins_0_20": bins,
        "global_threshold_over_mean_pfa_0_01": float(global_threshold_over_mean),
        "observed_peak_over_mean_0_20": observed_over_mean,
        "welch_validation_segments": int(pseg.shape[1]),
        "segment_detection_rate": segment_detection_rate,
        "fft_noise_floor_cv": fft_noise_cv,
        "welch_noise_floor_cv": welch_noise_cv,
        "amplitude": par["amplitude"],
        "phase": par["phase"],
```

```
    "rmse_residual": float(np.sqrt(np.mean(residual**2))),
    "amplitude_ci95_conditional": np.quantile(amp_draw, [0.025, 0.975]).tolist(),
    "phase_ci95_conditional": np.quantile(phase_draw, [0.025, 0.975]).tolist(),
    "residual_max_abs_acf_lag_1_20": float(np.max(np.abs(residual_acf))),
    "residual_jarque_bera_stat": float(jb.statistic),
    "residual_jarque_bera_p": float(jb.pvalue),
    "split_first_amplitude": first_params[0]["amplitude"],
    "split_second_amplitude": second_params[0]["amplitude"],
    "split_first_phase": first_params[0]["phase"],
    "split_second_phase": second_params[0]["phase"],
}

# Figure 1: data evidence for Q1.
fig, ax = plt.subplots(2, 2, figsize=(14, 8))
mask_t = t <= 5
ax[0, 0].plot(t[mask_t], x[mask_t], lw=0.65)
ax[0, 0].set(title="(a) 单源信号前5 s", xlabel="时间 / s", ylabel="幅值 / a.u.")
m = (f > 0) & (f <= 20)
ax[0, 1].semilogy(f[m], p[m], lw=0.7)
ax[0, 1].axvline(f_refined, color="r", ls="--", label=f"f={f_refined:.6f} Hz")
ax[0, 1].set(title="(b) 全段FFT功率谱", xlabel="频率 / Hz", ylabel="功率")
ax[0, 1].legend()
mw = (fw > 0) & (fw <= 20)
ax[1, 0].semilogy(fw[mw], pw[mw], color="tab:green", lw=0.8)
ax[1, 0].axvline(fw[wp], color="r", ls="--", label=f"f={fw[wp]:.6f} Hz")
ax[1, 0].set(title="(c) Welch功率谱（L=8192）", xlabel="频率 / Hz", ylabel="PSD")
ax[1, 0].legend()
zf = (f >= 1.8) & (f <= 2.2)
zw = (fw >= 1.8) & (fw <= 2.2)
ax[1, 1].plot(f[zf], p[zf] / p[zf].max(), label="FFT")
ax[1, 1].plot(fw[zw], pw[zw] / pw[zw].max(), label="Welch")
ax[1, 1].axvline(f_refined, color="r", ls="--")
ax[1, 1].set(title="(d) 2 Hz附近归一化谱", xlabel="频率 / Hz", ylabel="归一化功率")
ax[1, 1].legend()
for a in ax.ravel():
    a.grid(alpha=0.25)
fig.tight_layout()
fig.savefig(FIG / "图1.png", dpi=220)
plt.close(fig)

# Figure 2: direct, data-based comparison; no incompatible P_D values are mixed.
fstft, tstft, pstft = spectrogram(
    x - x.mean(), fs=FS, window="hann", nperseg=256, noverlap=128, mode="psd"
)
pstft_mean = pstft.mean(axis=1)
stft_peak = int(np.argmax(pstft_mean[1:]) + 1)
rows = [
    {
        "method": "STFT",
        "L": 256,
        "delta_f": FS / 256,
        "segments": int(len(tstft)),
        "peak_over_median": float(pstft_mean[stft_peak] / np.median(pstft_mean[1:])),
        "noise_floor_cv": np.nan,
        "segment_detection_rate": np.nan,
        "global_pfa_control": "否",
    },
    {
        "method": "全段FFT",
        "L": len(x),
        "delta_f": FS / len(x),
        "segments": 1,
        "peak_over_median": float(p[peak] / np.median(p[1:])),
        "noise_floor_cv": fft_noise_cv,
        "segment_detection_rate": np.nan,
        "global_pfa_control": "否",
    },
    {
        "method": "Welch",
        "L": 8192,
        "delta_f": FS / 8192,
        "segments": 8,
        "peak_over_median": float(pw[wp] / np.median(pw[1:])),
        "noise_floor_cv": welch_noise_cv,
        "segment_detection_rate": segment_detection_rate,
        "global_pfa_control": "与最大谱线检验联合后为是",
    },
]
pd.DataFrame(rows).to_csv(TABLE / "q1_method_comparison.csv", index=False)
fig, ax = plt.subplots(1, 2, figsize=(12, 4.8))
ax[0].bar([r["method"] for r in rows], [r["delta_f"] for r in rows])
```

```
ax[0].set_yscale("log")
ax[0].set(title="频率分辨率", ylabel=r"$\Delta f$ / Hz")
ax[1].bar([r["method"] for r in rows], [r["peak_over_median"] for r in rows])
ax[1].set_yscale("log")
ax[1].set(title="实测峰值/谱中位基底", ylabel="比值")
for a in ax:
    a.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig(FIG / "图2.png", dpi=220)
plt.close(fig)

# Q2 figures.
fig, ax = plt.subplots(3, 1, figsize=(14, 9), sharex=True)
mask = t <= 3
ax[0].plot(t[mask], x[mask], lw=0.6, alpha=0.7, label="观测信号")
ax[0].plot(t[mask], fitted[mask], color="r", lw=1.6, label="恢复周期分量")
ax[0].legend()
ax[0].set(title="观测信号与恢复周期分量", ylabel="幅值 / a.u.")
ax[1].plot(t[mask], fitted[mask], color="r")
ax[1].set(title="恢复周期分量", ylabel="幅值 / a.u.")
ax[2].plot(t[mask], residual[mask], color="0.45", lw=0.65)
ax[2].set(title=f"残差（RMSE={result['rmse_residual']:.4f}）", xlabel="时间 / s", ylabel="幅值 / a.u.")
for a in ax:
    a.grid(alpha=0.25)
fig.tight_layout()
fig.savefig(FIG / "图3.png", dpi=220)
plt.close(fig)

fr, pr = fft_spectrum(t, residual)
ff, pf = fft_spectrum(t, fitted)
fig, ax = plt.subplots(1, 2, figsize=(13, 4.8))
m = (f > 0) & (f <= 16)
ax[0].semilogy(f[m], p[m], label="观测")
ax[0].semilogy(ff[m], pf[m], label="恢复")
ax[0].semilogy(fr[m], pr[m], label="残差", alpha=0.8)
ax[0].set(title="全频段对比", xlabel="频率 / Hz", ylabel="功率")
mz = (f >= 1.8) & (f <= 2.2)
ax[1].semilogy(f[mz], p[mz], label="观测")
ax[1].semilogy(ff[mz], pf[mz], label="恢复")
ax[1].semilogy(fr[mz], pr[mz], label="残差")
ax[1].set(title="2 Hz附近", xlabel="频率 / Hz", ylabel="功率")
for a in ax:
    a.grid(alpha=0.25)
    a.legend()
fig.tight_layout()
fig.savefig(FIG / "图4.png", dpi=220)
plt.close(fig)

# Q2 controlled truth experiment. The clean sinusoid is known, while
# noise is sampled from the centered attachment residual so that its
# scale and non-parametric marginal distribution match the observed data.
sim_rng = np.random.default_rng(20260623)
n_sim = 500
clean = par["amplitude"] * np.sin(2 * np.pi * f_refined * t + par["phase"])
clean_rms = float(np.sqrt(np.mean(clean**2)))
sim_rows = []
for _ in range(n_sim):
    y_sim = clean + sim_rng.choice(residual_centered, size=len(t), replace=True)
    fsim, psim = fft_spectrum(t, y_sim)
    local = (fsim >= f_refined - 0.02) & (fsim <= f_refined + 0.02)
    local_ids = np.flatnonzero(local)
    k = int(local_ids[np.argmax(psim[local])])
    # Three-bin parabolic interpolation of the FFT peak gives a fast
    # continuous-frequency refinement for the 500-run validation.
    if 0 < k < len(psim) - 1:
        denom = psim[k - 1] - 2 * psim[k] + psim[k + 1]
        delta = 0.5 * (psim[k - 1] - psim[k + 1]) / denom if denom != 0 else 0.0
        delta = float(np.clip(delta, -0.5, 0.5))
    else:
        delta = 0.0
    fhat_sim = float(fsim[k] + delta * (fsim[1] - fsim[0]))
    p_sim, fit_sim, _, _ = ls_at_frequencies(t, y_sim, [fhat_sim])
    amp_error = abs(p_sim[0]["amplitude"] - par["amplitude"]) / par["amplitude"]
    phase_delta = np.angle(np.exp(1j * (p_sim[0]["phase"] - par["phase"])))
    sim_rows.append(
        {
            "frequency_abs_error_hz": abs(fhat_sim - f_refined),
            "amplitude_relative_error": amp_error,
            "phase_abs_error_rad": abs(phase_delta),
            "waveform_nrmse": float(np.sqrt(np.mean((fit_sim - clean) ** 2)) / clean_rms),
        }
```

```
        )
    sim_df = pd.DataFrame(sim_rows)
    sim_df.to_csv(TABLE / "q2_truth_simulation.csv", index=False)
    sim_summary = {
        "n": n_sim,
        "noise_source": "centered attachment residual resampling",
    }
    for col in sim_df.columns:
        sim_summary[col] = {
            "median": float(sim_df[col].median()),
            "mean": float(sim_df[col].mean()),
            "q025": float(sim_df[col].quantile(0.025)),
            "q975": float(sim_df[col].quantile(0.975)),
        }
    result["truth_simulation"] = sim_summary
    return result


def q3(t, x):
    f, p = fft_spectrum(t, x)
    noise_median = np.median(p[1:])
    peaks, props = find_peaks(p, height=5 * noise_median, distance=2)
    peaks = peaks[np.argsort(p[peaks])[::-1]]
    top = peaks[:10]

    bic_rows = []
    fixed_models = {}
    for m in range(1, 11):
        freqs = np.sort(f[top[:m]])
        params, fitted, residual, rss = ls_at_frequencies(t, x, freqs)
        # Conservatively count frequency, sine and cosine parameters: k_M=3M.
        bic = len(x) * np.log(rss / len(x)) + 3 * m * np.log(len(x))
        bic_rows.append({"M": m, "RSS": rss, "BIC": bic})
        fixed_models[m] = (freqs, params, fitted, residual)
    m_opt = int(min(bic_rows, key=lambda r: r["BIC"])["M"])
    initial = fixed_models[m_opt][0]
    refined = refine_multiple_frequencies(t, x, initial, half_width=0.005)
    params, fitted, residual, rss = ls_at_frequencies(t, x, refined)
    params = sorted(params, key=lambda z: z["frequency"])

    # Conditional residual bootstrap for BIC order-selection stability.
    # Candidate frequencies remain fixed; the experiment tests stability to residual noise.
    n_boot = 500
    model_q = []
    for m in range(1, 11):
        freqs_m = np.sort(f[top[:m]])
        cols = []
        for fm in freqs_m:
            cols.extend([np.sin(2 * np.pi * fm * t), np.cos(2 * np.pi * fm * t)])
        phi_m = np.column_stack(cols)
        q_m, _ = np.linalg.qr(phi_m, mode="reduced")
        model_q.append(q_m)
    base_fit = fixed_models[m_opt][2]
    base_residual = fixed_models[m_opt][3] - np.mean(fixed_models[m_opt][3])
    selected_orders = []
    batch_size = 20
    for start in range(0, n_boot, batch_size):
        b = min(batch_size, n_boot - start)
        ids = RNG.integers(0, len(x), size=(len(x), b))
        yb = base_fit[:, None] + base_residual[ids]
        y_energy = np.sum(yb**2, axis=0)
        scores = []
        for m, q_m in enumerate(model_q, 1):
            projection = q_m.T @ yb
            rss_b = np.maximum(y_energy - np.sum(projection**2, axis=0), 1e-15)
            scores.append(len(x) * np.log(rss_b / len(x)) + 3 * m * np.log(len(x)))
        selected_orders.extend((np.argmin(np.vstack(scores), axis=0) + 1).tolist())
    bootstrap_counts = {m: int(selected_orders.count(m)) for m in range(1, 11)}
    pd.DataFrame(
        [{"M": m, "count": bootstrap_counts[m], "proportion": bootstrap_counts[m] / n_boot}
         for m in range(1, 11)]
    ).to_csv(TABLE / "q3_bootstrap_order.csv", index=False)

    pd.DataFrame(bic_rows).to_csv(TABLE / "q3_bic.csv", index=False)
    pd.DataFrame(params).to_csv(TABLE / "q3_parameters.csv", index=False)

    # CRLB with unknown phase: centered-time Fisher information approximation.
    sigma = float(np.std(residual))
    a_ref = 0.025
    duration = float(t[-1] - t[0])
    crlb_std_hz = np.sqrt(
```

```
    24 * sigma**2 / (a_ref**2 * (2 * np.pi) ** 2 * len(x) * duration**2)
)

# Empirical two-peak resolution experiment.
deltas = np.array([0.0025, 0.005, 0.0075, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5])
n_mc = 500
resolution_rows = []
n = len(t)
df = FS / n
local_freq = np.fft.rfftfreq(n, 1 / FS)
local_mask = (local_freq >= 12.7) & (local_freq <= 13.8)
local_ids = np.flatnonzero(local_mask)
for delta in deltas:
    success = 0
    amp_errors = []
    for _ in range(n_mc):
        ph1, ph2 = RNG.uniform(-np.pi, np.pi, 2)
        y = (
            0.025 * np.sin(2 * np.pi * 13.0 * t + ph1)
            + 0.010 * np.sin(2 * np.pi * (13.0 + delta) * t + ph2)
            + RNG.normal(0, sigma, n)
        )
        spec = np.abs(np.fft.rfft(y) / n) ** 2
        lp, _ = find_peaks(spec[local_ids], distance=1)
        if len(lp) < 2:
            continue
        chosen = local_ids[lp[np.argsort(spec[local_ids[lp]])[-2:]]]
        est = np.sort(local_freq[chosen])
        tol = max(2.2 * df, 0.25 * delta)
        if abs(est[0] - 13.0) <= tol and abs(est[1] - (13.0 + delta)) <= tol:
            pp, *_ = ls_at_frequencies(t, y, est)
            aa = np.array(sorted([v["amplitude"] for v in pp], reverse=True))
            err = np.mean(np.abs(aa - np.array([0.025, 0.010])) / np.array([0.025, 0.010]))
            amp_errors.append(err)
            success += 1
    phat = success / n_mc
    z = 1.959963984540054
    denom = 1.0 + z**2 / n_mc
    center = (phat + z**2 / (2 * n_mc)) / denom
    half = z * np.sqrt(phat * (1 - phat) / n_mc + z**2 / (4 * n_mc**2)) / denom
    resolution_rows.append(
        {
            "delta_f": float(delta),
            "success_rate": phat,
            "success_wilson95_low": float(center - half),
            "success_wilson95_high": float(center + half),
            "mean_amplitude_relative_error": float(np.mean(amp_errors)) if amp_errors else np.nan,
            "n_mc": n_mc,
        }
    )
pd.DataFrame(resolution_rows).to_csv(TABLE / "q3_resolution.csv", index=False)

# Figures 5--7.
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
m = (f > 0) & (f <= 25)
ax[0].semilogy(f[m], p[m], lw=0.65)
for idx, par in enumerate(params):
    ax[0].axvline(par["frequency"], color="r", ls="--", alpha=0.7)
    ax[0].text(
        par["frequency"],
        0.96 - 0.14 * (idx % 2),
        f"{par['frequency']:.4f} Hz",
        rotation=90,
        ha="center",
        va="top",
        transform=ax[0].get_xaxis_transform(),
        fontsize=9,
    )
ax[0].set(title="多源FFT候选峰", xlabel="频率 / Hz", ylabel="功率")
ax[1].plot([r["M"] for r in bic_rows], [r["BIC"] for r in bic_rows], marker="o")
ax[1].axvline(m_opt, color="r", ls="--", label=f"M*={m_opt}")
ax[1].set(title="BIC模型阶数选择", xlabel="模型阶数 M", ylabel="BIC")
ax[1].legend()
for a in ax:
    a.grid(alpha=0.25)
fig.tight_layout()
fig.savefig(FIG / "图5.png", dpi=220)
plt.close(fig)

mask = t <= 3
fig, ax = plt.subplots(len(params) + 1, 1, figsize=(14, 12), sharex=True)
```

```
    components = []
    for i, par in enumerate(params):
        comp = par["amplitude"] * np.sin(2 * np.pi * par["frequency"] * t + par["phase"])
        components.append(comp)
        ax[i].plot(t[mask], x[mask], color="0.8", lw=0.5)
        ax[i].plot(t[mask], comp[mask], color="r", lw=1.2)
        ax[i].set_ylabel(f"{par['frequency']:.3f} Hz")
        ax[i].grid(alpha=0.2)
    ax[-1].plot(t[mask], x[mask], color="0.75", lw=0.5, label="观测")
    ax[-1].plot(t[mask], np.sum(components, axis=0)[mask], color="r", lw=1.1, label="联合恢复")
    ax[-1].set(xlabel="时间 / s", ylabel="总和")
    ax[-1].legend()
    ax[-1].grid(alpha=0.2)
    fig.tight_layout()
    fig.savefig(FIG / "图6.png", dpi=220)
    plt.close(fig)

    rr = pd.DataFrame(resolution_rows)
    fig, ax1 = plt.subplots(figsize=(10, 5.5))
    ax1.semilogx(rr.delta_f, rr.success_rate, marker="o", color="tab:blue")
    ax1.fill_between(
        rr.delta_f,
        rr.success_wilson95_low,
        rr.success_wilson95_high,
        color="tab:blue",
        alpha=0.18,
        label="Wilson 95%区间",
    )
    ax1.axhline(0.95, color="r", ls="--", label="95%可靠分离标准")
    ax1.set(xlabel=r"频率间距 $\Delta f$ / Hz", ylabel="双峰识别成功率", ylim=(0, 1.05))
    ax2 = ax1.twinx()
    ax2.semilogx(rr.delta_f, 100 * rr.mean_amplitude_relative_error, marker="s", color="tab:orange")
    ax2.set_ylabel("成功试验中的平均振幅相对误差 / %")
    ax1.grid(alpha=0.25)
    fig.tight_layout()
    fig.savefig(FIG / "图7.png", dpi=220)
    plt.close(fig)

    return {
        "N": len(x),
        "mean": float(np.mean(x)),
        "std": float(np.std(x)),
        "candidate_count_5x_median": int(len(peaks)),
        "M_opt": m_opt,
        "delta_BIC_5_minus_4": float(bic_rows[4]["BIC"] - bic_rows[3]["BIC"]),
        "bootstrap_order_n": n_boot,
        "bootstrap_order_counts": bootstrap_counts,
        "bootstrap_order_m4_proportion": bootstrap_counts[4] / n_boot,
        "parameters": params,
        "rmse": float(np.sqrt(np.mean(residual**2))),
        "crlb_frequency_std_unknown_phase_hz": float(crlb_std_hz),
        "resolution": resolution_rows,
    }


def q4(q3_result, sigma_base):
    # The geometry below is an explicit illustrative scenario, not supplied by the problem.
    width, height = 4.0, 2.0
    source_positions = np.array([[0.5, 1.0], [1.5, 0.3], [2.5, 1.7], [3.5, 1.0]])
    source_frequencies = np.array([4.0, 8.0, 13.0, 14.0])
    q3_map = {round(p["frequency"]): p["amplitude"] for p in q3_result["parameters"]}
    source_amplitudes = np.array([q3_map[int(f)] for f in source_frequencies])

    xs = np.arange(0.25, width, 0.25)
    ys = np.arange(0.25, height, 0.25)
    candidates = np.array([(x, y) for x in xs for y in ys])
    assert len(candidates) == 105

    alpha = 0.5
    t_obs = 4.0
    n_obs = int(FS * t_obs)
    n_monitored = len(source_positions)

    def per_source_pfa(system_pfa):
        """Bonferroni allocation, valid without assuming independent frequency tests."""
        return system_pfa / n_monitored

    def noise_sigma(points, scale=1.0):
        # Representative non-uniform motor-noise field, strongest near the device center.
        r2 = (points[:, 0] - 2.0) ** 2 + (points[:, 1] - 1.0) ** 2
        return sigma_base * scale * (1.0 + 0.35 * np.exp(-r2 / (2 * 0.6**2)))
```

```
def rho_matrix(alpha_value=alpha, noise_scale=1.0, amplitudes=source_amplitudes):
    d = np.linalg.norm(candidates[:, None, :] - source_positions[None, :, :], axis=2)
    sig = noise_sigma(candidates, noise_scale)[:, None]
    return (amplitudes[None, :] ** 2 * np.exp(-2 * alpha_value * d)) / (2 * sig**2)

def pd_from_lambda(lam, q, system_pfa=PFA):
    threshold = chi2.ppf(1 - per_source_pfa(system_pfa), 2 * q)
    return ncx2.sf(threshold, 2 * q, lam)

def optimize_q(q, rho):
    best_score = -1.0
    best_ids = None
    best_rho = None
    for ids in itertools.combinations(range(len(candidates)), q):
        combined = rho[list(ids)].sum(axis=0)
        score = float(combined.min())
        if score > best_score:
            best_score, best_ids, best_rho = score, ids, combined
    lam = n_obs * best_rho
    pds = pd_from_lambda(lam, q)
    return {
        "q": q,
        "ids": list(best_ids),
        "positions": candidates[list(best_ids)].tolist(),
        "rho_by_source": best_rho.tolist(),
        "rho_min": float(best_rho.min()),
        "snr_min_db": float(10 * np.log10(best_rho.min())),
        "pd_by_source": pds.tolist(),
        "pd_min": float(pds.min()),
    }

def evaluate_ids(ids, q, rho):
    combined = rho[list(ids)].sum(axis=0)
    pds = pd_from_lambda(n_obs * combined, q)
    return {
        "q": q,
        "ids": list(ids),
        "positions": candidates[list(ids)].tolist(),
        "rho_by_source": combined.tolist(),
        "rho_min": float(combined.min()),
        "snr_min_db": float(10 * np.log10(combined.min())),
        "pd_by_source": pds.tolist(),
        "pd_min": float(pds.min()),
    }

rho = rho_matrix()
layouts_scenario_specific = [optimize_q(q, rho) for q in [1, 2, 3]]

# The problem does not map the four detected amplitudes to four physical positions.
# Design conservatively for the weakest amplitude, strongest attenuation/noise,
# and the lower sensitivity bound, then evaluate under the nominal scenario.
weakest = float(source_amplitudes.min())
gain_min = 0.85
robust_rho = gain_min**2 * rho_matrix(
    alpha_value=0.8,
    noise_scale=1.2,
    amplitudes=np.full(4, weakest),
)
robust_designs = [optimize_q(q, robust_rho) for q in [1, 2, 3]]
layouts = [evaluate_ids(d["ids"], d["q"], rho) for d in robust_designs]
best = layouts[-1]

# Robustness over attenuation, noise scale and all frequency-position assignments.
sensitivity_rows = []
for a in [0.3, 0.4, 0.5, 0.6, 0.8]:
    for ns in [0.8, 1.0, 1.2]:
        rr = rho_matrix(
            alpha_value=a,
            noise_scale=ns,
            amplitudes=np.full(4, weakest),
        )
        # Evaluate the same robust layout; do not re-optimize after perturbation.
        sol = evaluate_ids(best["ids"], 3, rho_matrix(alpha_value=a, noise_scale=ns))
        sensitivity_rows.append(
            {
                "alpha": a,
                "noise_scale": ns,
                "positions": json.dumps(sol["positions"], ensure_ascii=False),
                "snr_min_db": sol["snr_min_db"],
                "pd_min": sol["pd_min"],
```

```
            }
        )
pd.DataFrame(sensitivity_rows).to_csv(TABLE / "q4_sensitivity.csv", index=False)

# Worst amplitude-to-position permutation evaluated for the baseline layout.
perm_pd = []
for amps in itertools.permutations(source_amplitudes):
    r = rho_matrix(amplitudes=np.asarray(amps))[best["ids"]].sum(axis=0)
    perm_pd.append(float(pd_from_lambda(n_obs * r, 3).min()))

# Monte Carlo calibration at orthogonal matched-filter-output level.
# Four simultaneous detectors are simulated to verify family-wise P_FA.
pfa_single = per_source_pfa(PFA)
threshold = chi2.ppf(1 - pfa_single, 6)
mc_n = 100000
h0_stats = RNG.chisquare(6, size=(mc_n, n_monitored))
empirical_pfa = float(np.mean(np.any(h0_stats > threshold, axis=1)))
empirical_pd = []
for lam in n_obs * np.asarray(best["rho_by_source"]):
    phase = RNG.uniform(-np.pi, np.pi, size=(mc_n, 3))
    lam_sensor = np.full(3, lam / 3.0)
    means = np.stack(
        [np.sqrt(lam_sensor)[None, :] * np.cos(phase),
         np.sqrt(lam_sensor)[None, :] * np.sin(phase)],
        axis=2,
    )
    z = RNG.normal(size=(mc_n, 3, 2)) + means
    empirical_pd.append(float(np.mean(np.sum(z**2, axis=(1, 2)) > threshold)))

# Measurement-position sensitivity is not supplied by the problem. Keep g_ij=1
# in the baseline and quantify ±15% multiplicative uncertainty for the selected layout.
selected_base_rho = rho[best["ids"]]
gain_pd = []
for _ in range(5000):
    gains = RNG.uniform(0.85, 1.15, size=selected_base_rho.shape)
    combined = (selected_base_rho * gains**2).sum(axis=0)
    gain_pd.append(float(pd_from_lambda(n_obs * combined, 3).min()))

# Figure 8: scenario and optimized three-sensor layout.
gx, gy = np.meshgrid(np.linspace(0, width, 161), np.linspace(0, height, 81))
gp = np.column_stack([gx.ravel(), gy.ravel()])
gd = np.linalg.norm(gp[:, None, :] - source_positions[None, :, :], axis=2)
gs = noise_sigma(gp)[:, None]
field = np.sum(source_amplitudes[None, :] ** 2 * np.exp(-2 * alpha * gd) / (2 * gs**2), axis=1)
field = 10 * np.log10(field.reshape(gx.shape))
fig, ax = plt.subplots(figsize=(10, 5.2))
c = ax.contourf(gx, gy, field, levels=30, cmap="YlOrRd")
fig.colorbar(c, ax=ax, label="四源累积线性SNR / dB")
ax.scatter(source_positions[:, 0], source_positions[:, 1], marker="*", s=230, c="red", edgecolor="k", label="假设故障源")
bp = np.asarray(best["positions"])
ax.scatter(bp[:, 0], bp[:, 1], marker="^", s=180, c="cyan", edgecolor="k", label="最优传感器")
for i, (xy, f0, aa) in enumerate(zip(source_positions, source_frequencies, source_amplitudes), 1):
    ax.text(xy[0] + 0.04, xy[1] + 0.06, f"S{i}: {f0:.0f} Hz, A={aa:.3f}")
ax.set(xlim=(0, width), ylim=(0, height), xlabel="x / m", ylabel="y / m", title="代表性4 m×2 m场景中的三传感器布局")
ax.legend(loc="lower right")
fig.tight_layout()
fig.savefig(FIG / "图8.png", dpi=220)
plt.close(fig)

# Figure 9: ROC and amplitude-scale performance.
pfas = np.logspace(-5, 0, 180)
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
for sol in layouts:
    r = np.asarray(sol["rho_by_source"])
    curve = [float(pd_from_lambda(n_obs * r, sol["q"], p).min()) for p in pfas]
    ax[0].semilogx(pfas, curve, label=f"{sol['q']}个传感器")
ax[0].axvline(PFA, color="k", ls="--")
ax[0].set(xlabel=r"系统总 $P_{FA}^{sys}$", ylabel=r"系统最差 $P_D$", title="ROC曲线", ylim=(0, 1.02))
scales_db = np.linspace(-12, 12, 100)
for sol in layouts:
    r = np.asarray(sol["rho_by_source"])
    curve = [float(pd_from_lambda(n_obs * r * 10 ** (db / 10), sol["q"]).min()) for db in scales_db]
    ax[1].plot(scales_db, curve, label=f"{sol['q']}个传感器")
ax[1].axvline(0, color="k", ls="--", label="问题三估计振幅")
ax[1].set(xlabel="故障功率尺度变化 / dB", ylabel=r"系统最差 $P_D$", title=r"固定系统总 $P_{FA}=0.01$", ylim=(0, 1.02))
for a in ax:
    a.grid(alpha=0.25)
    a.legend()
fig.tight_layout()
fig.savefig(FIG / "图9.png", dpi=220)
plt.close(fig)
```

```
# Figure 10: independent numerical calibration of theory at detector-output level.
theory_pd = np.asarray(best["pd_by_source"])
xbar = np.arange(n_monitored)
fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(xbar - 0.18, theory_pd, 0.36, label="非中心卡方理论值")
ax.bar(xbar + 0.18, empirical_pd, 0.36, label="蒙特卡洛值")
ax.set_xticks(xbar, [f"源{i}" for i in range(1, n_monitored + 1)])
ax.set(
    ylabel=r"$P_D$",
    ylim=(0, 1.02),
    title=rf"理论与蒙特卡洛校准（系统 $P_{{FA}}$={PFA:.2f}，经验值={empirical_pfa:.4f}）",
)
ax.legend()
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig(FIG / "图10.png", dpi=220)
plt.close(fig)

# Figure 11: observation-time effect with a consistent sample count.
times = np.unique(np.r_[np.logspace(0, 3, 80), 4, 400])
fig, ax = plt.subplots(figsize=(9, 5))
for sol in layouts:
    r = np.asarray(sol["rho_by_source"])
    curve = [float(pd_from_lambda(int(round(FS * tt)) * r, sol["q"]).min()) for tt in times]
    ax.semilogx(times, curve, label=f"{sol['q']}个传感器")
ax.axvline(4, color="k", ls="--", label="4 s实时场景")
ax.axvline(400, color="0.5", ls=":", label="400 s完整数据")
ax.set(xlabel="观测时长 / s", ylabel=r"系统最差 $P_D$", ylim=(0, 1.02), title=r"固定 $P_{FA}=0.01$")
ax.grid(alpha=0.25)
ax.legend()
fig.tight_layout()
fig.savefig(FIG / "图11.png", dpi=220)
plt.close(fig)

time_to_target = {}
for sol in layouts:
    r = np.asarray(sol["rho_by_source"])
    time_to_target[str(sol["q"])] = {}
    for target in [0.5, 0.9]:
        lo, hi = 0.01, 5000.0
        for _ in range(70):
            mid = 0.5 * (lo + hi)
            value = float(pd_from_lambda(max(1, int(round(FS * mid))) * r, sol["q"]).min())
            if value >= target:
                hi = mid
            else:
                lo = mid
        time_to_target[str(sol["q"])][str(target)] = hi

# Figure 12: false-alarm sensitivity, generated from the same layouts.
pfa_levels = np.array([0.001, 0.005, 0.01, 0.02, 0.05, 0.1])
xloc = np.arange(len(pfa_levels))
fig, ax = plt.subplots(figsize=(10, 5))
width_bar = 0.23
pfa_rows = []
for k, sol in enumerate(layouts):
    r = np.asarray(sol["rho_by_source"])
    vals = [float(pd_from_lambda(n_obs * r, sol["q"], p).min()) for p in pfa_levels]
    ax.bar(xloc + (k - 1) * width_bar, vals, width_bar, label=f"{sol['q']}个传感器")
    for p, v in zip(pfa_levels, vals):
        pfa_rows.append({"q": sol["q"], "P_FA": p, "P_D_min": v})
ax.set_xticks(xloc, [str(v) for v in pfa_levels])
ax.set(xlabel=r"系统总 $P_{FA}^{sys}$", ylabel=r"系统最差 $P_D$", ylim=(0, 1.02), title="系统虚警率约束敏感性")
ax.grid(axis="y", alpha=0.25)
ax.legend()
fig.tight_layout()
fig.savefig(FIG / "图12.png", dpi=220)
plt.close(fig)
pd.DataFrame(pfa_rows).to_csv(TABLE / "q4_pfa.csv", index=False)
pd.DataFrame(layouts).to_json(TABLE / "q4_layouts.json", orient="records", force_ascii=False, indent=2)

return {
    "scenario_notice": "4m×2m geometry, source locations and spatial noise field are illustrative assumptions, not problem-given data.",
    "width": width,
    "height": height,
    "source_positions": source_positions.tolist(),
    "source_frequencies": source_frequencies.tolist(),
    "source_amplitudes_from_q3": source_amplitudes.tolist(),
    "candidate_count": len(candidates),
    "combination_count_q3": int(len(list(itertools.combinations(range(len(candidates)), 3)))),
```

```
        "alpha": alpha,
        "noise_model": "sigma(p)=sigma0*[1+0.35 exp(-||p-(2,1)||^2/(2*0.6^2))]",
        "sigma0": sigma_base,
        "observation_time_primary_s": t_obs,
        "system_pfa": PFA,
        "per_source_pfa_bonferroni": float(pfa_single),
        "per_source_pfa_sidak_independence_reference": float(1.0 - (1.0 - PFA) ** (1.0 / n_monitored)),
        "layouts": layouts,
        "layouts_scenario_specific": layouts_scenario_specific,
        "layout_policy": "robust design uses weakest amplitude, alpha=0.8, noise scale=1.2 and g_min=0.85 at every potential source position",
        "permutation_worst_pd_for_baseline_q3_layout": float(min(perm_pd)),
        "permutation_best_pd_for_baseline_q3_layout": float(max(perm_pd)),
        "empirical_pfa": empirical_pfa,
        "empirical_pd_by_source": empirical_pd,
        "time_to_target_pd_s": time_to_target,
        "measurement_sensitivity_model": "baseline g_ij=1; robustness test g_ij independently varies in [0.85,1.15]",
        "measurement_sensitivity_pd_quantiles_5_50_95": np.quantile(gain_pd, [0.05, 0.5, 0.95]).tolist(),
    }


def main():
    t1, x1 = load_sheet(0)
    t3, x3 = load_sheet(1)
    q12 = q1_q2(t1, x1)
    q3_result = q3(t3, x3)
    q4_result = q4(q3_result, q12["rmse_residual"])
    results = {"q1_q2": q12, "q3": q3_result, "q4": q4_result}
    with open(OUT / "results.json", "w", encoding="utf-8") as fh:
        json.dump(results, fh, ensure_ascii=False, indent=2)
    print(json.dumps(results, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```